

SHAREX - RIDE BOOKING SYSTEM WITH VIRTUAL ASSISTANT

CLOUD COMPUTING

Submitted by

BHUVANESHWARI K (230701055)

ISHITHA S (230701117)

VETRISSELVI M (230701379)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

**COMPUTER SCIENCE AND
ENGINEERING**



**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING RAJALAKSHMI ENGINEERING
COLLEGE**

MAY 2026

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report titled “**SHAREX - RIDE BOOKING SYSTEM WITH VIRTUAL ASSISTANT**” is the bonafide work of **BHUVANESHWARI K (230701055)**, **ISHITHA S (230701117)**, **VETRISIELVI M (230701379)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. E. M Malathy
Professor &
Head of The Department
Department of Computer Science
and Engineering
Rajalakshmi Engineering College

SIGNATURE

Dr.M.Ayyadurai
Supervisor
Associate Professor
Department of Computer Science
and Engineering
Rajalakshmi Engineering College

Submitted to Project Viva Voce Examination held on_____

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S.MEGANATHAN, B.E, F.I.E.,** our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.,** and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.,** for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.,** our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr.MALATHY E.M, Ph.D.,** Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **DR. M. AYYADURAI M.E.,Ph.D.,** Associate Professor, Department of Computer Science and Engineering for his valuable guidance throughout the course of the project.

**BHUVANESHWARI K
ISHITHA S
VETRISELVI M**

ABSTRACT

In today's rapidly growing urban environment, transportation has become a significant challenge for daily commuters. Individuals frequently face issues such as traffic congestion, high travel costs, unreliable ride availability, and lack of efficient real-time tracking systems. These challenges not only increase travel time but also reduce overall convenience and productivity. While ride-hailing platforms exist to address these concerns, developing a scalable and efficient system remains a complex task without proper technological support. This project aims to develop a ride sharing application that connects riders and drivers through a centralized digital platform, enabling efficient and reliable transportation services. The system allows users to book rides, view ride details, and interact with the application in a seamless manner. By automating the ride allocation process, the platform reduces manual effort and improves overall efficiency in managing transportation services. The application is designed using a cloud-based architecture, ensuring scalability, flexibility, and high availability. The system supports essential functionalities such as user registration, secure login, ride booking, and basic ride tracking. In addition, a chatbot interface is integrated to assist users through text-based commands, enabling easy interaction for booking rides and retrieving information. The backend of the system handles core operations such as user authentication, ride management, and data processing, while the frontend provides a user-friendly interface for smooth navigation and interaction. A secure database is used to store user and ride information, ensuring data consistency and reliability. The system also incorporates proper access control and monitoring mechanisms to maintain security and performance. The overall design follows a modular and scalable approach, allowing the system to handle increasing numbers of users without performance degradation. The deployment strategy ensures reliable operation with minimal downtime, making the application suitable for real-world usage scenarios. The expected outcome of this project is a cost-effective, scalable, and efficient ride sharing platform that improves transportation accessibility, reduces waiting time, and enhances user experience. By leveraging modern cloud-based technologies, the system demonstrates an effective solution for addressing real-world transportation challenges.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1	Introduction	1
1.1	Problem Statement	2
1.2	Objective of the Project	4
1.3	Scope and Boundaries	6
1.4	Stakeholders and End Users	8
1.5	Technologies Used	10
1.6	Organization of the Report	12
2	System Design and Architecture	14
2.1	Requirement Summary (Functional & Non-Functional)	15
2.2	Proposed Solution Overview	17
2.3	Cloud Deployment Strategy (AWS)	19
2.4	Infrastructure Requirements	21
2.5	AWS Services Mapping and Justification	23
3	Implementation	26
3.1	Frontend Development (HTML, CSS & JavaScript)	27
3.2	Serverless Backend Implementation (AWS Lambda)	29
3.3	Static Hosting & Deployment (AWS Amplify / S3)	31
3.4	API Integration & Data Flow	33
3.5	User Interface and Responsive Design	35
4	Cloud Operations and Security	38
4.1	AWS IAM Roles, Policies and Security	39
4.2	Monitoring and Logging (CloudWatch)	41
4.3	Authentication & Authorization	43
4.4	Data Management and API Integration	45
5	Results and Discussion	48
5.1	Implementation Summary	49

Chapter No.	Title	Page No.
5.2	Challenges Faced and Resolutions	51
5.3	Performance and Cost Observations	53
5.4	Key Learnings and Team Contributions	55
6	Conclusion and Future Enhancement	58
6.1	Conclusion	59
6.2	Future Scope	61
	References	63
	Appendices	65

CHAPTER 1 - INTRODUCTION

1.1 PROBLEM STATEMENT

In today's fast-paced urban environment, commuters face significant challenges in securing convenient, reliable, and affordable transportation. Traditional ride-booking methods and existing transport services often suffer from poor user experience, lack of transparency, and limited accessibility. These issues create inconvenience for both riders and drivers.

Passengers frequently struggle to find rides quickly, especially during peak hours or in suburban areas, due to the absence of a simple and efficient booking platform. Many existing systems have outdated interfaces, slow response times, and are not fully responsive across different devices, making the booking process frustrating and time-consuming. Additionally, users do not receive proper real-time updates on ride status, driver location, or estimated arrival time, which reduces trust and transparency in the service.

Without a well-designed digital platform, coordination between riders and drivers remains inefficient, resulting in longer waiting times and suboptimal route management. Security and trust concerns also persist due to the lack of proper user authentication mechanisms. Furthermore, most traditional systems lack a clean, modern, and intuitive interface with easy navigation and quick booking options.

These challenges collectively lead to decreased user satisfaction, lower adoption of digital ride-sharing services, and inefficient utilization of available transport resources.

To overcome these limitations, there is a clear need for a modern, user-friendly, visually appealing, and responsive ride-sharing web platform that provides a seamless booking experience.

1.2 OBJECTIVE OF THE PROJECT

The primary objective of this project is to design and implement a cloud-based ride sharing application that enables secure, efficient, and reliable transportation services by connecting riders and drivers through a centralized digital platform. The system aims to address challenges such as ride availability, high travel costs, lack of real-time tracking, and security concerns by integrating automation, scalability, and a user-friendly interface.

1. Improve Transportation Efficiency and Reduce Waiting Time:

Enable users to quickly book rides and get connected with nearby drivers, reducing waiting

time and improving overall travel efficiency. The system ensures optimal ride allocation to minimize delays and enhance user convenience.

2. Ensure Secure and Verified User Interaction:

Develop a platform where both riders and drivers are authenticated, ensuring safe and trustworthy interactions. The system incorporates secure login mechanisms and controlled access to protect user data and maintain reliability.

3. Enhance Cost Efficiency and Resource Utilization:

Provide an optimized ride matching system that reduces unnecessary travel distance and fuel consumption. This helps in lowering travel costs for users while improving earnings and efficiency for drivers.

4. Enable Real-Time Monitoring and Tracking:

Allow users to view ride details and receive updates in real-time. This improves transparency, builds trust, and enhances the overall user experience.

5. Integrate Intelligent Interaction and Automation:

Incorporate a chatbot system that supports text-based commands for booking rides and accessing information. This simplifies user interaction and reduces manual effort in navigating the application.

6. Ensure Scalability and High Performance:

Design the system using a cloud-based architecture that can handle a growing number of users without performance degradation. The platform is built to support high availability and efficient operation under varying loads.

1.3 SCOPE AND BOUNDARIES

The scope of this project includes the development and deployment of a cloud-based ride sharing application with integrated backend services and scalable infrastructure. The system covers key functionalities such as user authentication, ride booking and driver allocation.

The application supports core features including user registration and login, ride request management, and interaction through a text-based chatbot. The backend manages ride processing and data handling, while the frontend provides a user-friendly interface. A secure database is used to store user and ride information.

The system is deployed on a cloud platform to ensure scalability, reliability, and efficient performance. Monitoring and security mechanisms are implemented to maintain system integrity and protect user data. However, the project is limited to basic ride sharing functionalities. Advanced features such as full real-time GPS tracking, payment integration, and voice-based chatbot interaction are not included.

1.4 STAKEHOLDERS AND END USER

Stakeholders: Team Members – Responsible for designing, developing, testing, and maintaining the system modules, including user authentication, ride booking and management, chatbot interaction, database integration, and overall application functionality of the ShareX platform.

End Users:

- 1. **Riders:** who want to book rides quickly and travel conveniently from one location to another.
- 2. **Drivers:** who use the platform to accept ride requests and provide transportation services efficiently.
- 3. **New Users:** who seek an easy-to-use and reliable system for accessing transportation services with minimal effort.

1.5 TECHNOLOGIES USED

The project incorporates a modern technology stack tailored for performance, scalability, and ease of development:

Table 1. Frontend Technologies

Technology	Description
HTML5	Used for structuring the web pages.
CSS3 & Tailwind	For responsive and modern UI design with clean styling.
JavaScript (ES6)	For implementing dynamic functionality and interactivity.
Font Awesome	For icons and visual elements across the application.

Table 2: Backend Technologies

Technology	Description
AWS Lambda	Executes backend logic such as ride request handling and processing.
Amazon API Gateway	Acts as an entry point to receive and route API requests.
Amazon DynamoDB	Stores user data, ride requests, and ride history.
Amazon Cognito	Manages user authentication and secure login

Table 3: Cloud and Infrastructure

Technology / Service	Purpose
Amazon Web Services (AWS)	Primary cloud platform used to host, manage, and monitor all backend and frontend services.
AWS Amplify	Hosts the frontend application and integrates with GitHub for continuous deployment (CI/CD).
AWS Lambda	Executes backend functions in a serverless environment with automatic scaling.
Amazon API Gateway	Creates and manages REST APIs and routes client requests to backend services.
Amazon Cognito	Provides secure user authentication, authorization, and user management features.
Amazon DynamoDB	Stores ride details, user information, chatbot data, and application records in a scalable NoSQL database.
Amazon Lex	Powers the chatbot functionality using natural language processing and conversational AI.
Amazon CloudWatch	Monitors application performance, logs, metrics, and system activities in real time.
AWS CloudTrail	Tracks and records AWS account activities and API usage for auditing and security monitoring.
GitHub	Stores source code and integrates with AWS Amplify for version control and CI/CD workflow.

Table 4: AI and Machine Learning

AI / ML Service	Purpose
Amazon Lex	Provides the chatbot interface for ride booking, user queries, and interactive assistance.
AWS Lambda	Processes chatbot requests and integrates AI-driven responses with backend services and APIs.

Table 5: DevOps and CI/CD

DevOps / CI-CD Tool	Purpose
GitHub	Used for version control, collaboration, and storing the ShareX project source code.
AWS Amplify	Provides automatic build, deployment, and continuous integration whenever code is pushed from GitHub.
Amazon Web Services GUI (AWS Management Console)	Used to configure, monitor, and manage AWS cloud services through a graphical user interface.

1.6 ORGANIZATION OF THE REPORT

This report is structured to provide a complete and logical documentation of the entire project lifecycle.

Chapter 1 introduces the problem context, project objectives, scope, stakeholders, technologies used, and overall report structure.

Chapter 2 presents a detailed overview of the system design and architecture, including requirements, proposed solution, cloud deployment strategy, infrastructure details, and service mapping.

Chapter 3 focuses on the implementation aspects, including API integration, serverless architecture, and overall system development using cloud services.

Chapter 4 discusses operational aspects and security mechanisms, including access control, monitoring, logging, and data protection strategies.

Chapter 5 presents the results and discussion, including implementation summary, challenges faced, performance observations, and key learnings.

Chapter 6 concludes the report with a summary of the project and highlights possible future enhancements.

The document concludes with references and appendices containing additional details such as configurations and system outputs.

CHAPTER 2 - SYSTEM DESIGN AND ARCHITECTURE

2.1 REQUIREMENT SUMMARY

Functional Requirements

The system is built using a modern serverless architecture on Amazon Web Services (AWS). The system combines a responsive frontend with scalable cloud backend services to deliver a smooth and secure ride-booking experience.

The frontend of the application is developed using HTML5, CSS3, and Tailwind CSS, with JavaScript for dynamic interactions. It is hosted on AWS Amplify, which provides fast static hosting, automatic deployments from GitHub, and global content delivery.

The backend follows a serverless approach using AWS Lambda functions to handle core business logic such as ride request processing, booking management, and user interactions. All API requests are routed through Amazon API Gateway, which serves as the secure entry point between the frontend and backend services.

User data, ride bookings, and history are stored in Amazon DynamoDB, offering scalable and high-performance NoSQL database capabilities. User authentication, registration, and session management are securely handled by Amazon Cognito, ensuring safe login and identity verification.

Security and access control are managed through AWS IAM roles and policies. Monitoring and observability are implemented using Amazon CloudWatch for logging and metrics, while AWS CloudTrail tracks account activity and API calls for auditing and compliance.

The source code is maintained in GitHub, integrated with AWS Amplify to enable Continuous Integration and Continuous Deployment (CI/CD) of the frontend. Additionally, Amazon Lex is integrated to provide an intelligent chatbot interface for user assistance and simplified ride booking.

This combination of technologies enables ShareX to deliver a responsive, secure, and scalable ride-sharing platform without the overhead of managing traditional servers.

Non-Functional Requirements

The ShareX ride-sharing platform is designed to meet high standards of performance, usability, security, scalability, and cost-effectiveness. The key non-functional requirements are as follows:

The system ensures good performance with fast page load times and smooth user interactions. This is achieved through optimized HTML, CSS, and JavaScript code along with Tailwind CSS framework.

Hosting on AWS Amplify further improves delivery speed through its global Content Delivery Network (CDN).

Scalability is supported by the serverless architecture. AWS Lambda and Amazon API Gateway automatically scale based on user demand, allowing the system to handle multiple concurrent ride requests efficiently without manual server management.

High availability is ensured by leveraging AWS cloud infrastructure, which offers reliable uptime and fault tolerance. Users can access the platform consistently with minimal downtime.

Security is a core requirement of the system. Amazon Cognito is used for secure user authentication and authorization. AWS IAM roles and policies enforce strict access control, while Amazon API Gateway provides secure endpoints with proper validation. All sensitive operations are protected to maintain user trust and data privacy.

Data durability and reliability are maintained through Amazon DynamoDB, which automatically replicates data across multiple availability zones, ensuring data is safe and consistently available.

The system is designed for good maintainability. The modular architecture separates frontend, backend logic, database, and authentication services, making it easier to update, debug, and extend features in the future. Finally, cost efficiency is achieved by adopting a serverless model. Services such as AWS Lambda, Amplify, and DynamoDB follow a pay-as-you-go pricing model. For this academic project, the implementation stays well within the AWS Free Tier limits, resulting in near-zero operational cost while maintaining full functionality.

2.2 PROPOSED SOLUTION OVERVIEWS

The proposed solution is a cloud-native, service-oriented ride sharing platform designed to achieve scalability, flexibility, and efficient performance. The system is structured into multiple layers to separate concerns and ensure maintainability. At the presentation layer, a web or mobile interface provides a user-friendly experience for riders and drivers. This frontend communicates with backend services through a secure API layer.

The backend handles core business logic, including user authentication, ride booking, driver allocation, and ride management workflows. Data persistence is managed using a NoSQL database that stores information related to users, drivers, and ride history. The system ensures fast and reliable data access to support real-time operations.

An API management layer is used to handle communication between the frontend and backend services, ensuring secure and efficient request handling. The backend follows a serverless approach, allowing the system to automatically scale based on demand while reducing infrastructure management.

To enhance user interaction, a chatbot is integrated into the system, enabling users to perform actions such as booking rides and checking ride details through text-based commands. This improves accessibility and simplifies the overall user experience.

The system also incorporates monitoring and logging mechanisms to track performance and system activities. Security is enforced through proper authentication, access control, and data protection techniques to ensure safe and reliable operation. Overall, this architecture combines the benefits of cloud computing, serverless design, and modular development to deliver a scalable, efficient, and user-friendly ride sharing application.

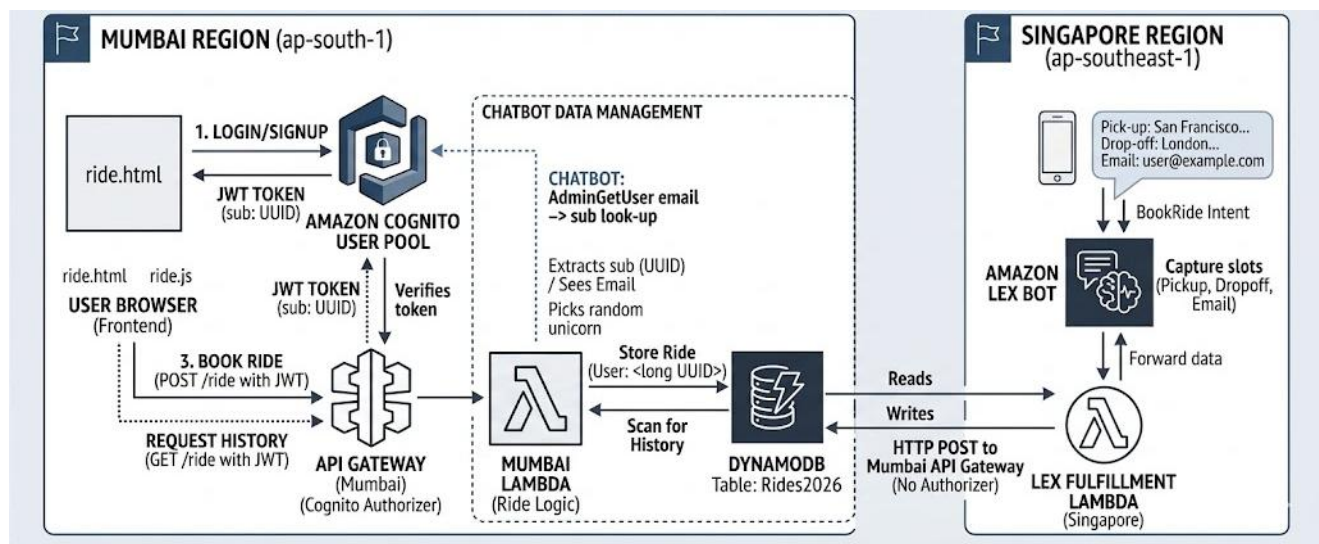


Fig 2.2.1 System Architecture

2.3 CLOUD DEPLOYMENT STATERGY

The deployment strategy leverages Amazon Web Services (AWS) as the primary cloud platform, utilizing a combination of managed and serverless services to minimize operational overhead while maximizing scalability and flexibility. The primary deployment region is Mumbai, selected for its proximity to the target user base and low latency, while selected services such as chatbot integration are configured in the Singapore region.

The architecture follows a modular, service-oriented pattern with serverless components. The frontend application is deployed using AWS Amplify, which provides fast and reliable hosting with

built-in deployment support. Backend logic is handled using AWS Lambda, enabling on-demand execution without the need for server management.

API communication between the frontend and backend is managed through Amazon API Gateway, which ensures secure and efficient request routing. Data storage is handled by Amazon DynamoDB, providing high performance and scalability for storing user, driver, and ride-related data. A chatbot service is integrated using Amazon Lex, which processes user commands through text-based interaction. This service operates on-demand, ensuring efficient resource utilization.

The overall workflow begins with users interacting with the frontend interface. API requests are routed through API Gateway to the backend services, where ride processing and data operations are performed. The backend interacts with the database to store and retrieve information, while the chatbot service is invoked when user interaction is required.

Monitoring and logging are handled through integrated cloud services to track system performance and user activity. This layered architecture ensures clear separation of concerns, enabling independent scaling of components and efficient system performance.

2.4 INFRASTRUCTURE REQUIREMENTS

Compute Resources:

- **AWS Lambda:** Serverless compute service used to execute backend logic such as ride request processing, booking management, and business operations. Lambda functions are configured with adequate memory allocation (typically 256 MB to 512 MB) and appropriate timeout settings. The service automatically scales based on incoming traffic.
- **Amazon API Gateway:** Acts as the frontend for all backend APIs. It handles HTTP requests from the web application, performs request validation, and routes traffic to Lambda functions. It provides built-in throttling, security, and monitoring capabilities.

Storage Requirements:

- **Amazon DynamoDB:** Serves as the primary NoSQL database for the application. It is used to store user profiles, ride details, booking history, and other dynamic data. The project utilizes the DynamoDB Free Tier (25 GB storage and on-demand capacity mode), which automatically scales as per application demand.
- **AWS Amplify Hosting:** Used for static frontend hosting. It provides fast global delivery through Amazon CDN, secure hosting, and sufficient storage for all web assets (HTML, CSS, JavaScript, and images). It supports quick updates and rollbacks through

continuous deployment.

2.5 AZURE SERVICES MAPPING AND JUSTIFICATION

Table 6: Service Mapping

Requirement	AWS Service	SKU/Tier	Purpose	Justification	Est. Monthly Cost (₹)
Frontend Hosting	AWS Amplify	Free Tier	Host web application	Easy deployment with GitHub CI/CD integration	Free /~\$17
User Authentication	Amazon Cognito	Free Tier	Manage user login, signup and sessions	Secure and scalable authentication without custom code	Free
API Management	Amazon API Gateway	Free Tier	Handle and route API requests	Provides secure, scalable REST API endpoints	Free
Backend Logic	AWS Lambda	Free Tier	Execute backend operations	Serverless execution with automatic scaling	Free
Database	Amazon DynamoDB	Free Tier	Store user data, ride details and history	High performance, scalable NoSQL database	Free
Access Control & Security	AWS IAM	Free Tier	Manage permissions and secure access	Ensures secure communication between services	Free
Monitoring & Logging	Amazon CloudWatch	Free Tier	Monitor application performance and logs	Real-time monitoring, alerts, and troubleshooting	Free
Auditing & Compliance	AWS CloudTrail	Free Tier	Track API calls and user activity	Provides audit logs for security and compliance	Free
Chatbot / AI Assistant	Amazon Lex	Free Tier	Provide conversational interface	Enables intelligent chatbot for ride booking and user queries	Free /~\$3
Version Control & CI/CD	GitHub	Free Tier	Store source code and automate deployment	Seamless integration with AWS Amplify for CI/CD	Free

CHAPTER 3 - DEVOPS IMPLEMENTATION

3.1 CONTINUOUS INTEGRATION AND DEPLOYMENT STATERGY

The project uses AWS Amplify integrated with GitHub to automate the build and deployment of the frontend application. This setup enables continuous integration and continuous delivery (CI/CD) for the React-based frontend, ensuring that every code update pushed to the main branch is automatically built and deployed to the live environment without any manual intervention.

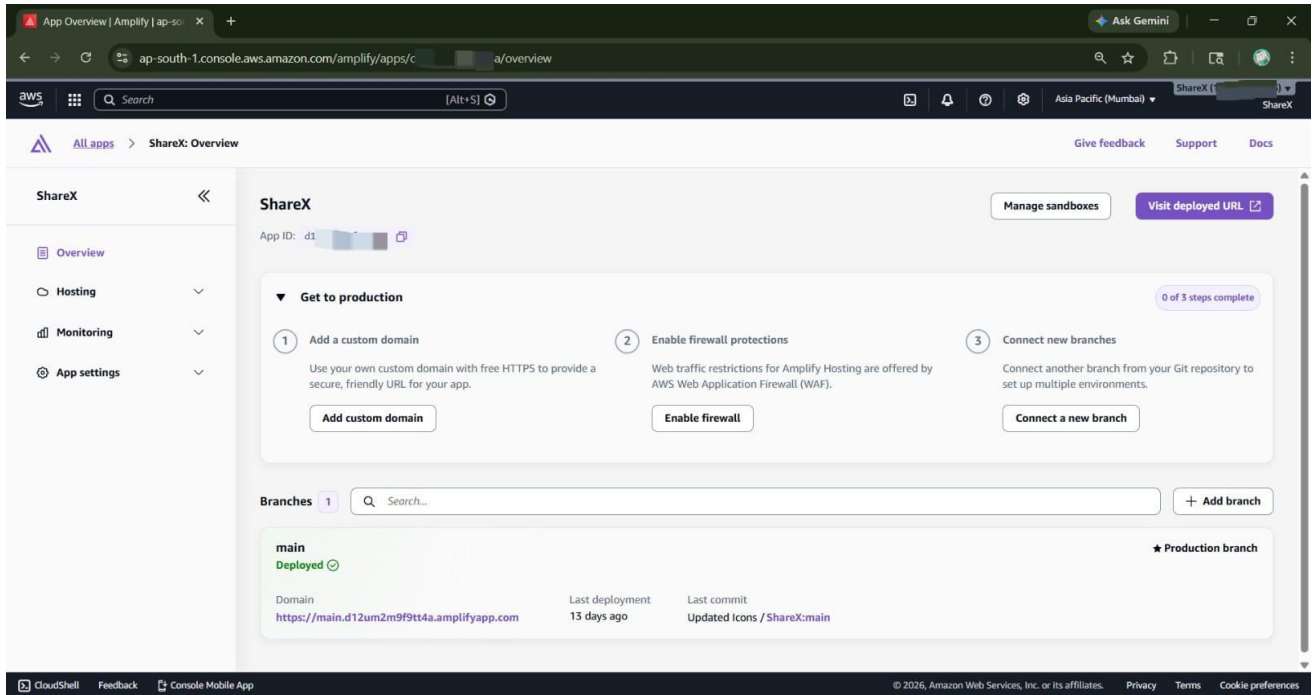


Figure 1. AWS Amplify Deployment Overview

The deployment setup follows a two-stage process:

- **Source Stage:** AWS Amplify connects to the GitHub repository. When code is pushed to the main branch, Amplify detects the change and automatically triggers a new build.
- **Build and Deploy Stage:** Amplify installs dependencies, builds the React application, and deploys the output to the hosted URL. Environment variables such as API Gateway URL and Cognito credentials are injected securely during the build.

The entire deployment process is triggered automatically on every push to the main branch. This eliminates manual deployment effort and ensures the live application always reflects the latest code changes.

Stage 1: Frontend Build and Deployment (AWS Amplify)

The frontend, built using HTML5, Tailwind CSS, and JavaScript, is hosted on **AWS Amplify**. The service monitors the main branch of the GitHub repository. On every commit and push, Amplify automatically:

- Pulls the latest source code
- Installs required dependencies
- Builds the static web application
- Deploys the updated version to the live URL

The application is currently deployed at: <https://main.d12um2m9f9tt4a.amplifyapp.com>

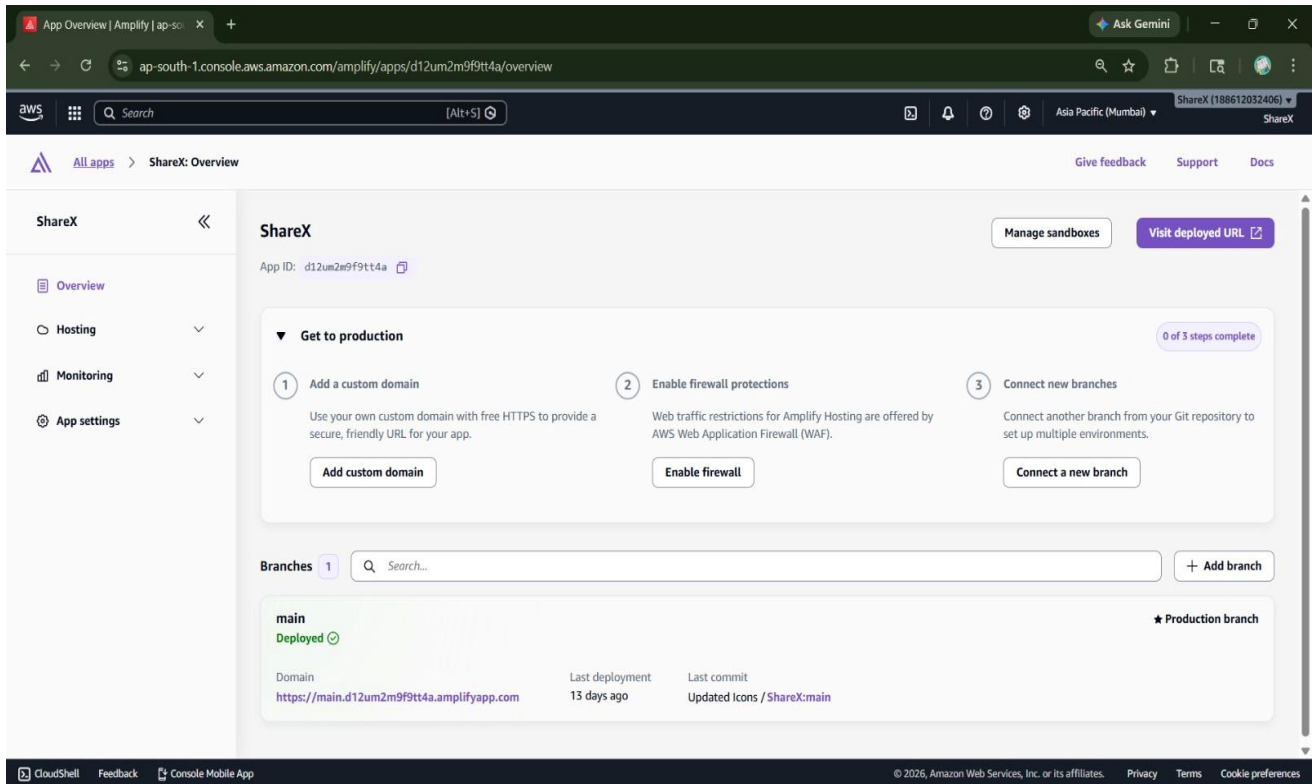


Figure 2. AWS Amplify App Overview - ShareX Deployed Application

Stage 2: Backend Deployment (AWS Lambda)

The backend logic is implemented using AWS Lambda functions. The primary function RequestCar2026 handles ride booking requests, user validation, and interactions with DynamoDB. These functions are deployed via the AWS Management Console and are triggered through Amazon API Gateway. This setup allows backend updates with zero downtime.

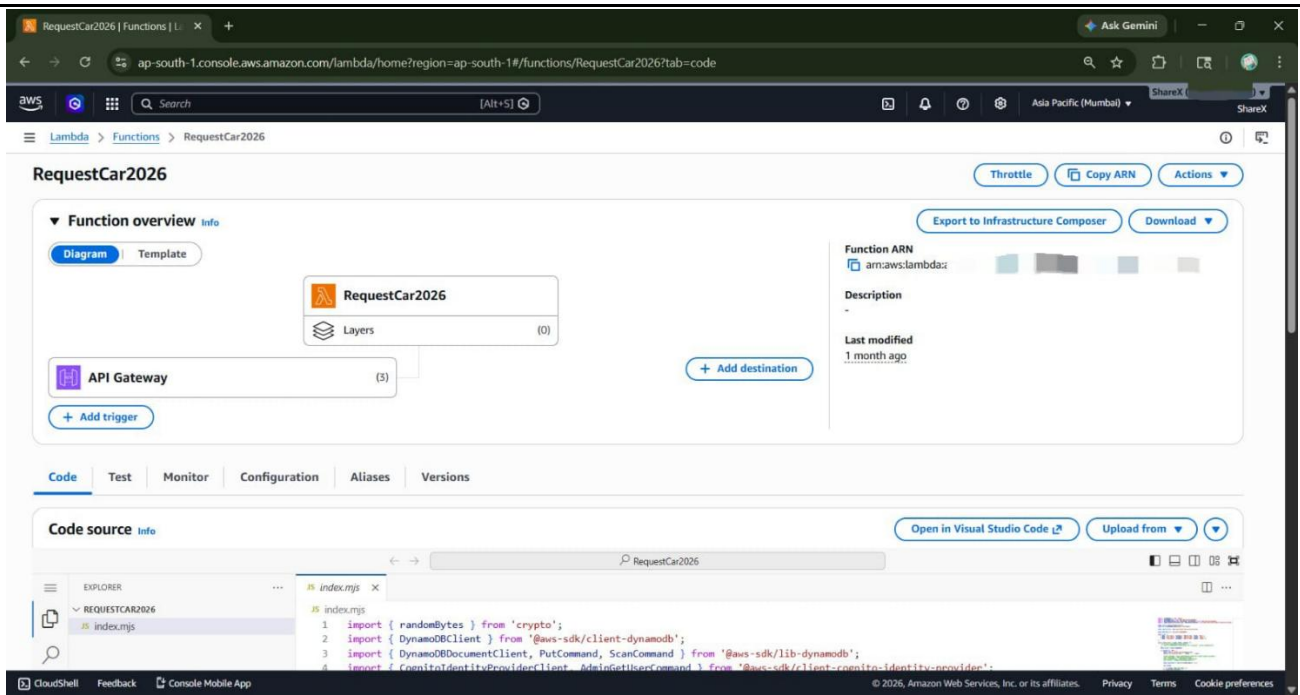


Figure 3. AWS Lambda Function - RequestCar2026

GitHub Secrets (Environment Variables)

The following secrets and environment variables are configured in the GitHub repository for secure CI/CD integration:

No.	Environment Variable	Purpose
1	AWS_ACCESS_KEY_ID	AWS authentication
2	AWS_SECRET_ACCESS_KEY	AWS authentication
3	AWS_REGION	AWS region configuration
4	API_GATEWAY_URL	Backend API endpoint
5	COGNITO_USER_POOL_ID	Cognito authentication
6	COGNITO_APP_CLIENT_ID	Cognito client ID
7	DYNAMODB_TABLE_NAME	DynamoDB table reference
8	LAMBDA_FUNCTION_NAME	Lambda function name
9	IAM_ROLE_ARN	IAM role for permissions
10	LEX_BOT_NAME	Amazon Lex chatbot integration

3.2 SERVERLESS CONFIGURATION AND CI/CD IMPLEMENTATION

The ShareX platform adopts a fully serverless architecture on AWS. Since the project does not use Infrastructure as Code (IaC) tools such as Terraform, all cloud resources were manually configured and managed through the **AWS Management Console**. This approach provided simplicity, faster development, and easy integration between services.

Core Infrastructure Components

Backend and Serverless Execution

- **AWS Lambda:** All backend business logic is implemented using AWS Lambda functions. The primary function (RequestCar2026) handles ride booking requests, user input validation, and data operations with DynamoDB. Lambda functions operate in an event-driven model and automatically scale according to the incoming request load.
- **Amazon API Gateway:** Serves as the secure API layer that receives requests from the frontend and routes them to the appropriate Lambda functions. It also handles request validation, throttling, and security policies.
- **Amazon DynamoDB:** Acts as the primary database for storing user information, ride requests, booking history, and other application data. It provides high availability, low latency, and automatic scaling.
- **Amazon Cognito:** Manages complete user authentication, including user registration, login, password management, and session handling.

CI/CD Implementation

The project implements Continuous Integration and Continuous Deployment (CI/CD) using **AWS Amplify** integrated with **GitHub**.

When developers push code to the main branch of the GitHub repository, the following automated process is triggered:

1. AWS Amplify detects the code changes.
2. It automatically builds the frontend (HTML, Tailwind CSS, and JavaScript).
3. Environment variables such as API Gateway URL, Cognito User Pool ID, and Cognito App Client ID are injected during the build process.
4. The built application is deployed to the live URL.

Backend updates to AWS Lambda functions are deployed separately through the AWS Management Console. All sensitive credentials and configuration values are securely stored using **GitHub Secrets**, ensuring they are not exposed in the source code.

This combination of serverless services and automated CI/CD pipeline allows rapid development, easy updates, zero server management, and consistent deployment of the ShareX ride-sharing platform.

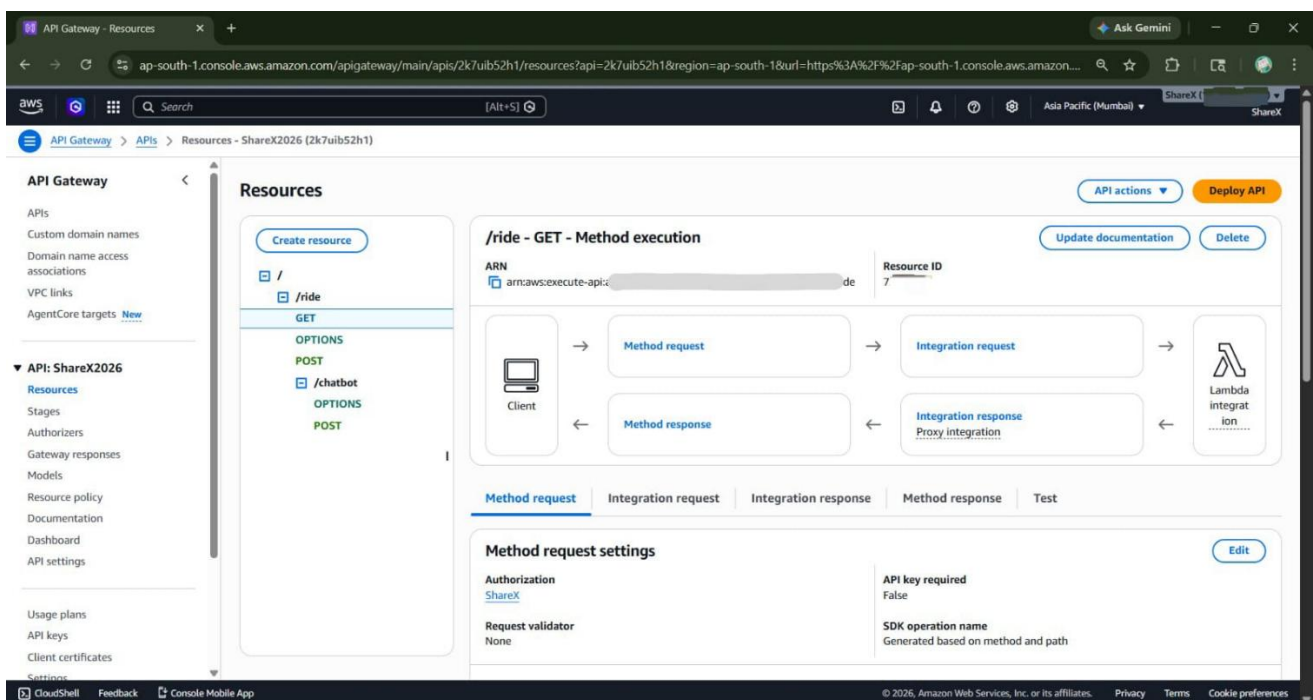
3.3 SERVERLESS EXECUTION STRATEGY

The system is designed using an **event-driven architecture** where each operation is executed independently. This model ensures high scalability and efficient resource utilization.

Execution Flow:

1. **Trigger Stage (Amazon API Gateway)** All client requests from the frontend are received by **Amazon API Gateway**. It acts as the secure entry point for the application and performs request validation, authentication checks, routing, and throttling.
2. **Execution Stage (AWS Lambda)** Once validated, API Gateway triggers the appropriate **AWS Lambda** function. The Lambda function executes the core business logic in an isolated environment. Key operations handled by Lambda include:
 - Processing ride booking requests
 - User data validation
 - Interacting with DynamoDB for data storage and retrieval
 - Returning appropriate responses to the frontend

Each Lambda function runs only when invoked and automatically scales based on the number of incoming requests. This eliminates the need for constant server uptime and optimizes cost, as payment is only for the actual compute time used.



Deployment, Scaling, and Versioning

1. Lambda Function Setup:

Resource Allocation: Each function is configured with memory ranging from 128 MB to 512 MB and execution timeout limits based on workload requirements, ensuring efficient resource utilization.

Auto-scaling: The system automatically scales by creating multiple

concurrent function instances based on incoming request load, eliminating the need for manual scaling configuration.

API Integration: External access is enabled through API Gateway, which ensures secure communication using HTTPS and efficient request routing.

2. **Function Versioning and Management:**

Versioning is handled within Lambda by maintaining multiple function versions. A structured versioning approach is followed, where the latest version is used for development and stable versions are retained for production deployment. This allows easy tracking of updates and supports rollback in case of failures.

3.4 **SERVERLESS ORCHESTRATION AND SCALING (AWS ARCHITECTURE CONTROL)**

The ShareX platform leverages the native orchestration and auto-scaling capabilities of AWS serverless services. This approach eliminates the need for managing servers, clusters, or containers while providing automatic scaling, high availability, and built-in resilience.

1. **System Architecture and Deployment Design**

The application follows a fully decoupled serverless architecture:

- **Frontend:** Hosted on AWS Amplify with global CDN support.
- **Backend:** Implemented using AWS Lambda functions.
- **API Layer:** Managed by Amazon API Gateway.
- **Database:** Amazon DynamoDB.

Amazon API Gateway serves as the central entry point (orchestrator) for all incoming requests. It handles routing, security, rate limiting, and request validation before forwarding traffic to the respective Lambda functions. Each Lambda function is mapped to specific operations such as ride booking, user management, or data retrieval.

Sensitive configurations are securely managed through **AWS IAM roles**, while non-sensitive settings are passed via environment variables.

2. **Scaling, Resilience, and Performance**

- **Automatic Scaling:** AWS Lambda automatically scales out by creating new execution instances based on incoming request volume. There is no manual intervention required for scaling. The system can handle sudden spikes in ride requests efficiently.
- **Resilience and Fault Tolerance:**
 - Lambda functions include built-in retry mechanisms for failed executions.
 - Amazon DynamoDB automatically replicates data across multiple Availability Zones for high durability.
 - Amazon API Gateway ensures reliable request handling with timeout and circuit breaker-like capabilities.

- **Performance Optimization:** The serverless design ensures low latency and efficient resource utilization. Cold starts are minimized through provisioned concurrency (if needed) and optimized function configuration.

This serverless orchestration model gives the ShareX application excellent scalability, reliability, and operational simplicity while keeping infrastructure costs minimal.

3.5 SERVERLESS INTELLIGENCE AND AWS SERVICE MAPPING

1. Intelligent Ride Processing (AWS Lambda + API Gateway Integration)

This feature uses AWS Lambda and API Gateway to process ride requests and convert user input into structured ride data for the ride-sharing system.

Implementation:

The logic is handled inside AWS Lambda functions. API Gateway receives the user request and forwards it to Lambda for processing. This allows serverless execution without managing infrastructure.

Process:

- API Gateway receives ride request from user
- Request is sent to AWS Lambda function
- Lambda extracts source, destination, and user details
- Structured ride data is generated
- Data is stored in Amazon DynamoDB

2. Secure Authentication and Content Validation (Amazon Cognito + AWS Services)

This service ensures secure user authentication and validates all ride requests before processing.

Implementation:

Amazon Cognito is used for user authentication, while API Gateway and Lambda handle request validation and processing. IAM ensures secure service-to-service communication.

Process:

- User logs in using Amazon Cognito
- Cognito generates JWT authentication token
- API Gateway verifies token before request processing
- Lambda validates ride request data
- Valid data is stored in DynamoDB

CHAPTER 4 - CLOUD OPERATIONS AND SECURITY

4.1 DEVSECOPS INTEGRATION

Security is integrated throughout the development and deployment lifecycle of the ShareX platform using AWS native services and best practices. Security measures are enforced at multiple layers including authentication, API access, data storage, and service permissions.

Amazon Cognito User Pool

User authentication and authorization are managed using Amazon Cognito. The User Pool `bdrt-n` handles user registration, login, and JWT token generation. Only authenticated users with valid tokens can access protected resources. Currently, the system has 2 registered users.

Amazon Cognito Identity Pool

The Identity Pool `ShareXIdentityPool` provides temporary AWS credentials to authenticated and unauthenticated users, enabling secure access to AWS services without exposing long-term credentials in the application.

AWS Key Management Service (KMS) AWS KMS is used for encryption of sensitive data and secrets. Customer-managed KMS keys are utilized to encrypt DynamoDB data at rest and to secure sensitive environment variables in AWS Lambda functions. KMS ensures that data remains protected and can only be accessed by authorized services through strict key policies, adding an extra layer of data security and compliance.

This multi-layered security approach (Cognito + IAM + KMS) ensures that user data, ride information, and system credentials remain secure throughout the application lifecycle.

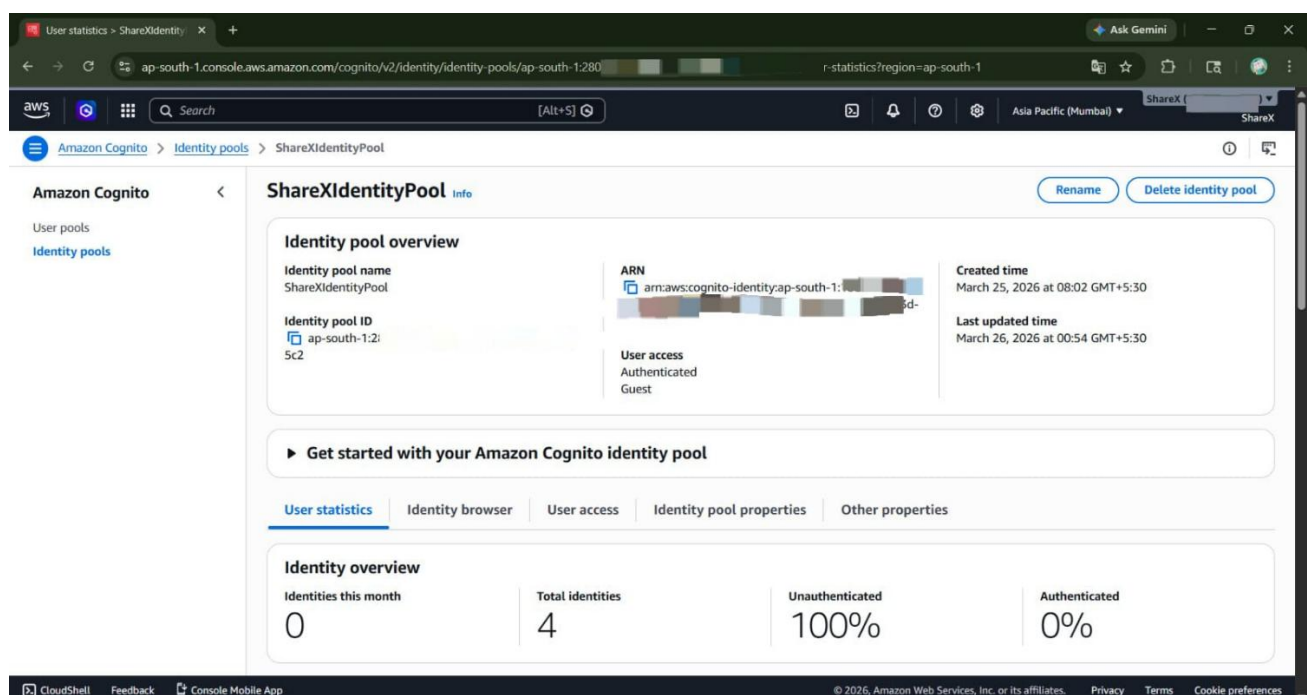


Figure 5.1. Amazon Cognito Identity Pool

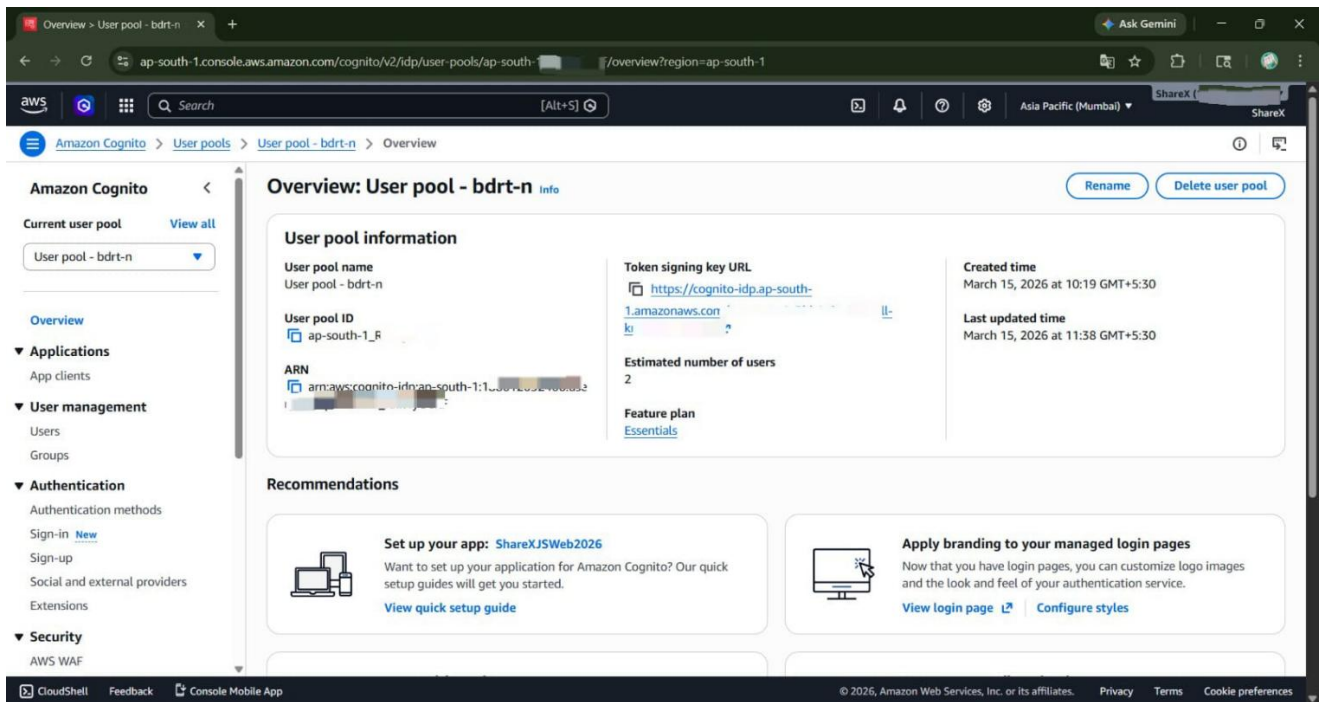


Figure 5.2. Amazon Cognito User Pool

4.2 MONITORING AND OBSERVABILITY

Comprehensive monitoring and observability are implemented using **Amazon CloudWatch** and **AWS CloudTrail**.

Amazon CloudWatch CloudWatch collects and stores logs from all Lambda functions. The log group /aws/lambda/RequestCar2026 contains 63 log streams with detailed execution information including duration, billed duration, errors, and request results. This enables effective debugging and performance monitoring.

AWS CloudTrail A multi-region trail named management-events is configured to record all AWS API calls made within the account. The trail stores logs in an S3 , providing a complete audit history of management and security-related activities.

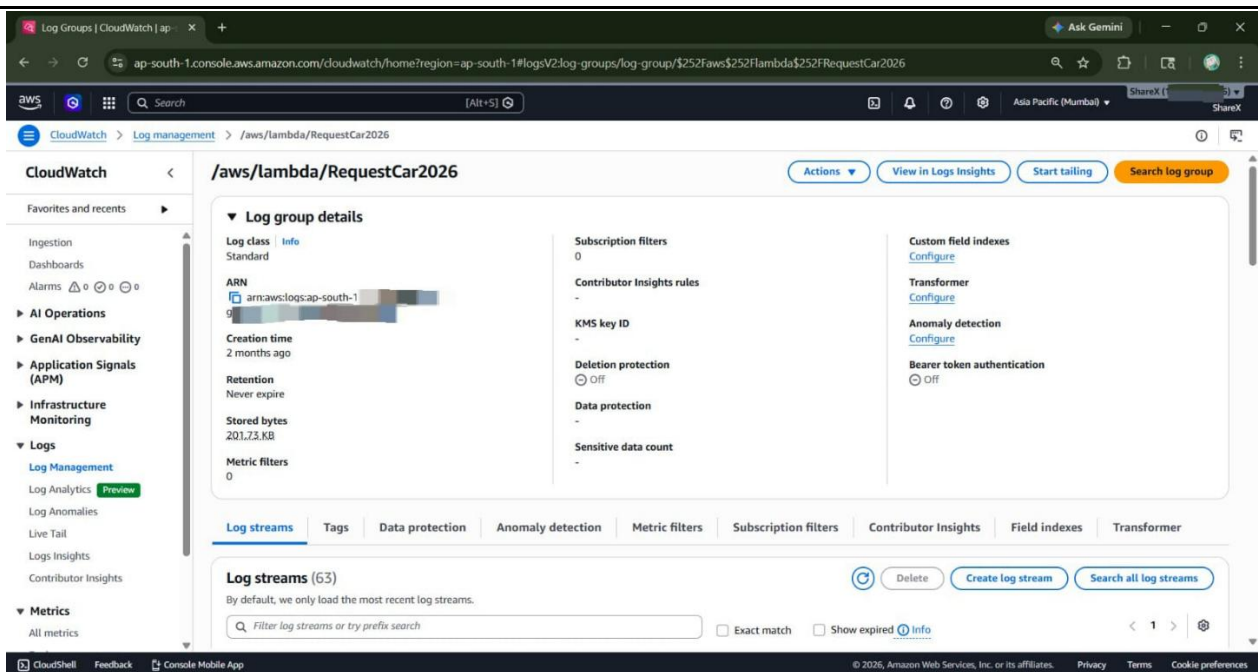


Figure 6. CloudWatch Log Group

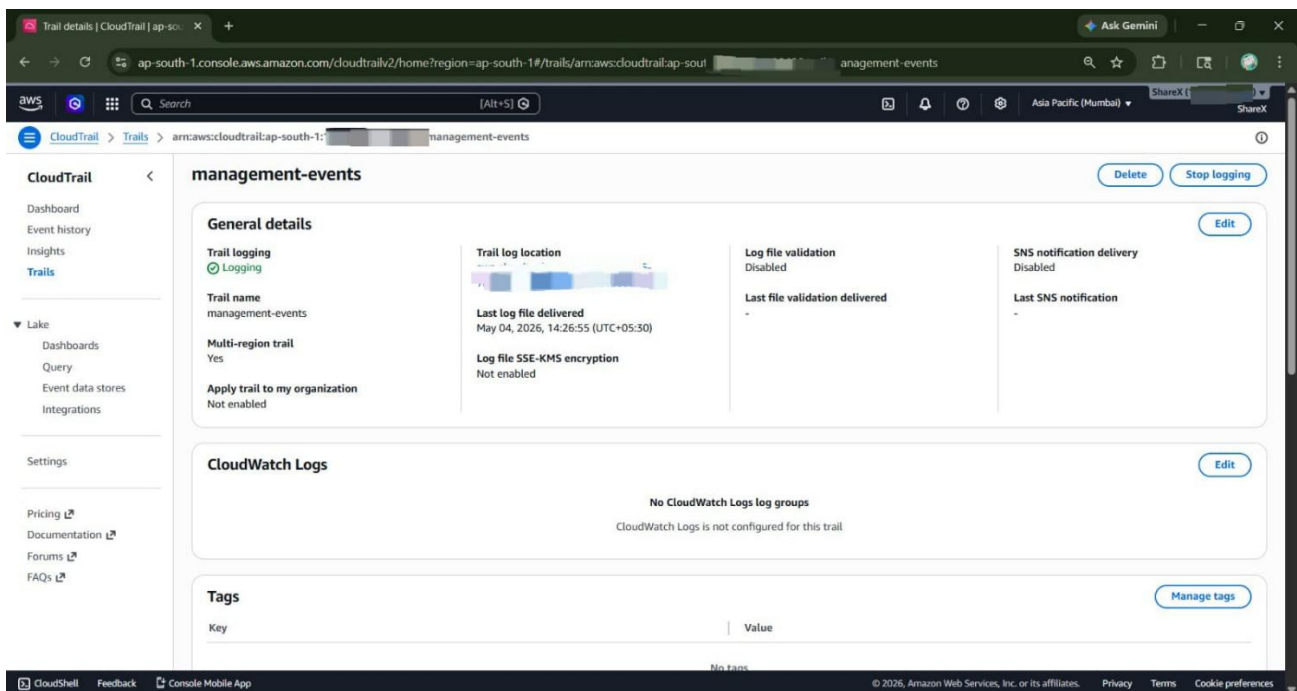


Figure 7. AWS CloudTrail – management

4.3 ACCESS CONTROL (IAM)

Access control is implemented following the **principle of least privilege** using **AWS Identity and Access Management (IAM)**.

The IAM dashboard shows 9 roles, 3 policies, 2 IAM users, and 1 user group. The root user has MFA enabled and no active access keys. All services operate through scoped IAM roles instead of root credentials.

Key IAM roles created for the project include:

- ShareXIdentityPoolAuthRole
- ShareXIdentityPoolUnauthRole
- ShareXLambda2026 (Lambda execution role)

The Lambda execution role is granted only the necessary permissions for DynamoDB access and CloudWatch logging.

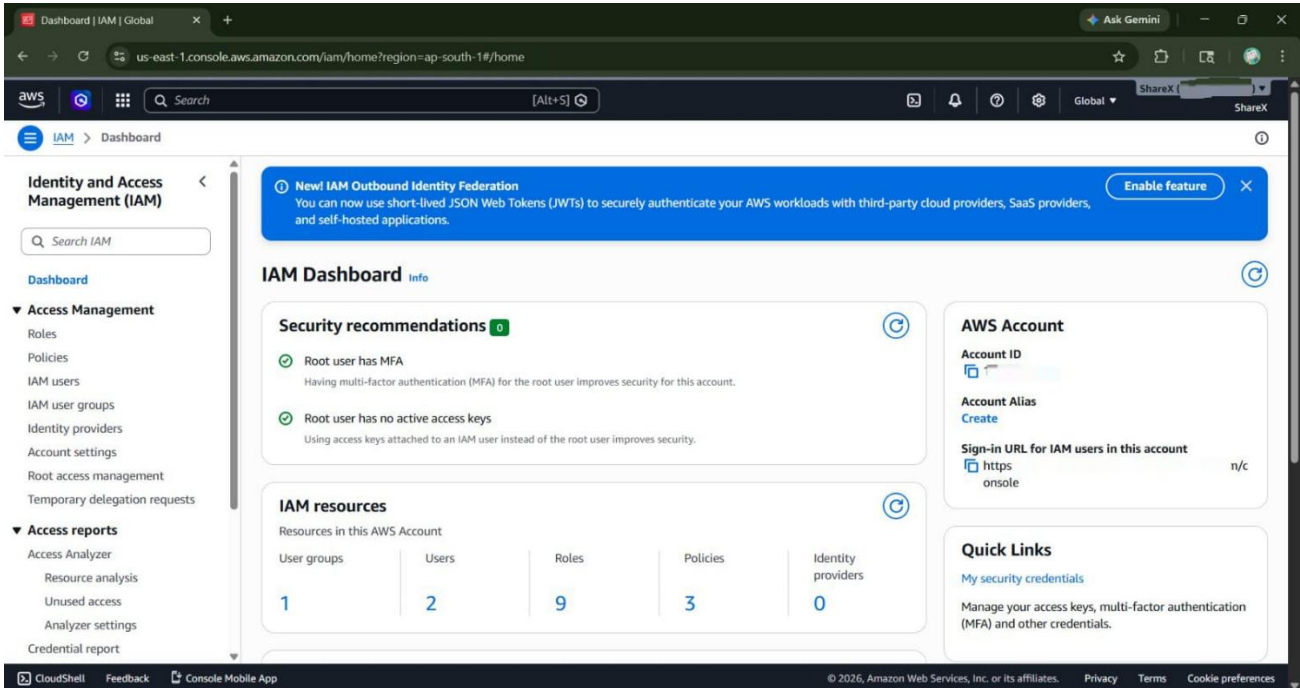


Figure 8. IAM Dashboard

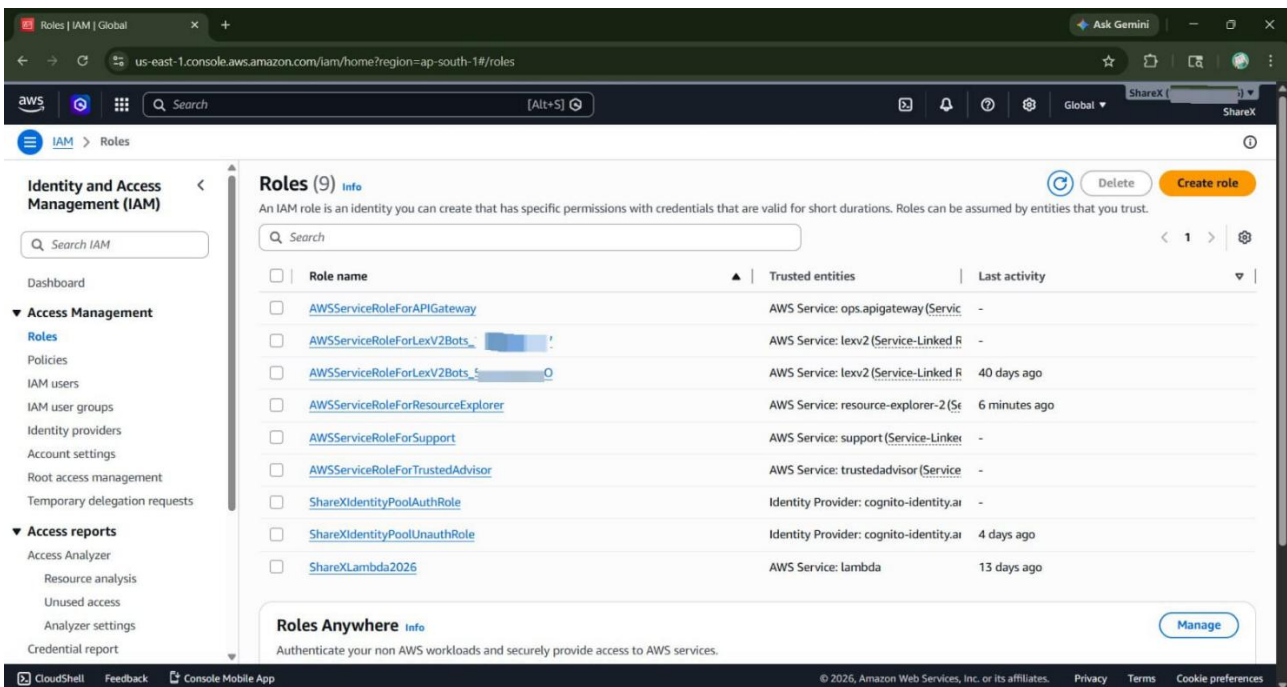


Figure 9. IAM Roles

4.4 Data Management and API Integration

The ShareX platform uses a combination of Amazon DynamoDB and Amazon API Gateway to efficiently manage data storage and secure communication between the frontend and backend.

Amazon DynamoDB All ride-related data is stored in the Rides2026 DynamoDB table. This table currently contains 11 ride records. The main attributes include RideId (String partition key), PickupName, DropoffName, RequestTime, Unicorn (vehicle details), and User (rider ID). The table operates in on-demand capacity mode, ensuring automatic scaling and cost efficiency. A recent table scan demonstrated high efficiency, consuming only 2 Read Capacity Units (RCUs).

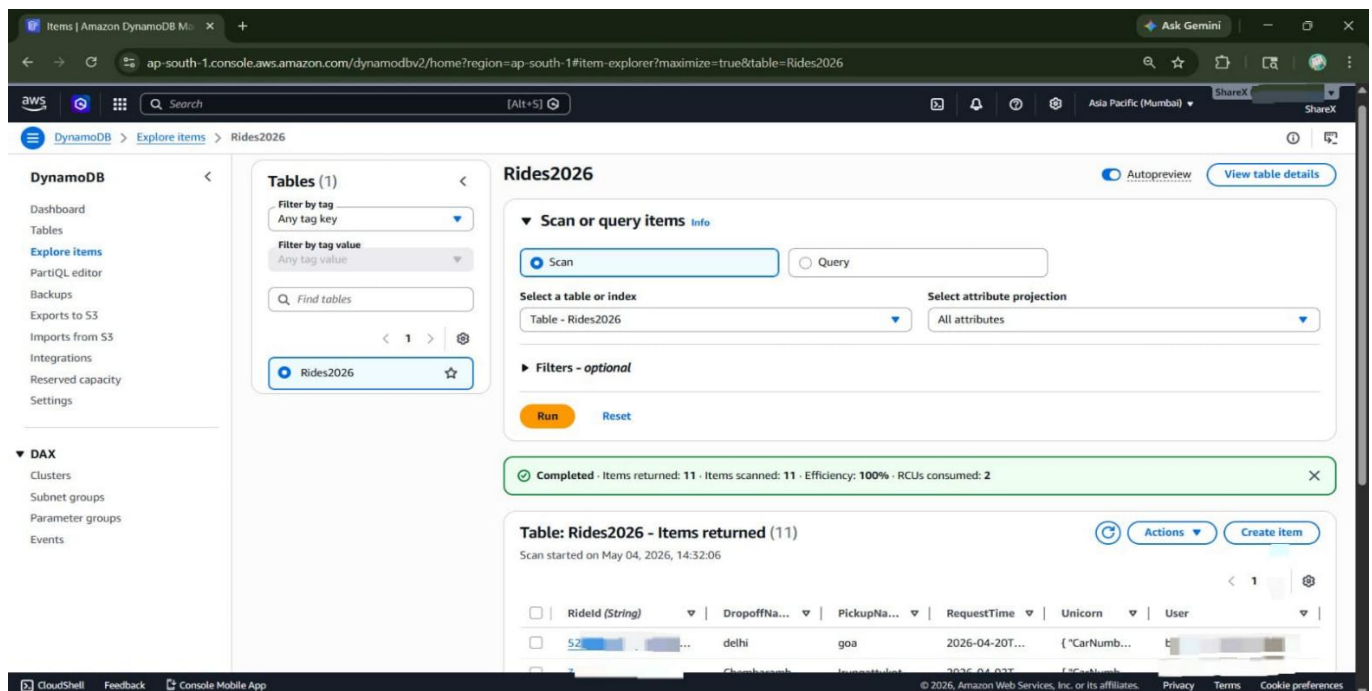


Figure 10. Amazon DynamoDB – Rides2026 Table (11 Records)

Amazon API Gateway Amazon API Gateway serves as the secure API management layer. The API named ShareX2026 exposes RESTful endpoints for the application. Key resources include:

- /ride – Supports GET, POST, and OPTIONS methods for ride booking and retrieval.
- /chatbot – Handles requests from the Amazon Lex chatbot.

All endpoints use Lambda proxy integration and are protected by the ShareX Cognito Authorizer. This ensures that only authenticated users with valid JWT tokens can access the APIs. API key authentication is disabled as Cognito handles all authorization.

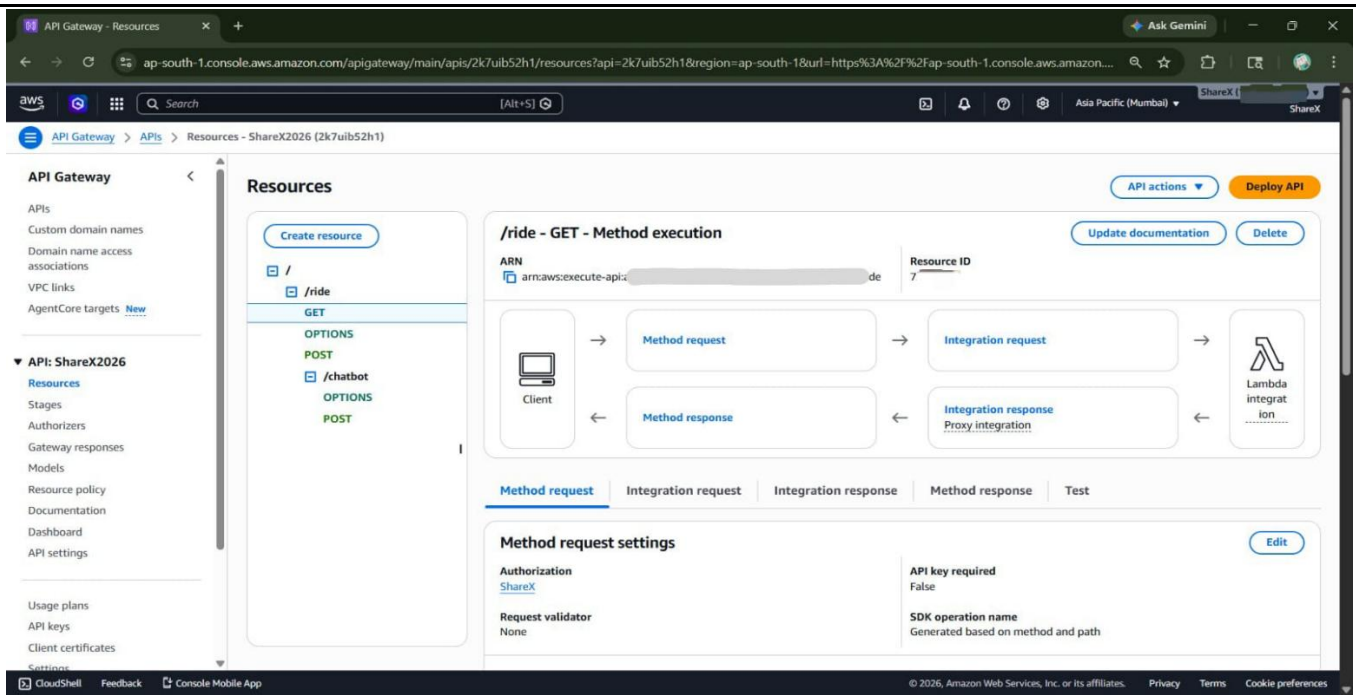


Figure 11. Amazon API Gateway – ShareX2026 Resources and Methods

4.5 Chatbot Integration and Cost Management

Amazon Lex Chatbot Integration To enhance user interaction, the platform integrates Amazon Lex for conversational ride booking. The bot named ShareXLexChatBot is deployed in the Asia Pacific (Singapore) region. It is configured with three intents:

- **BookRide** – For processing ride booking requests
- **Greeting** – For welcoming users and initial interaction
- **FallbackIntent** – For handling unrecognized user inputs

The chatbot processes natural language inputs from users and forwards structured data to AWS Lambda functions for backend processing.

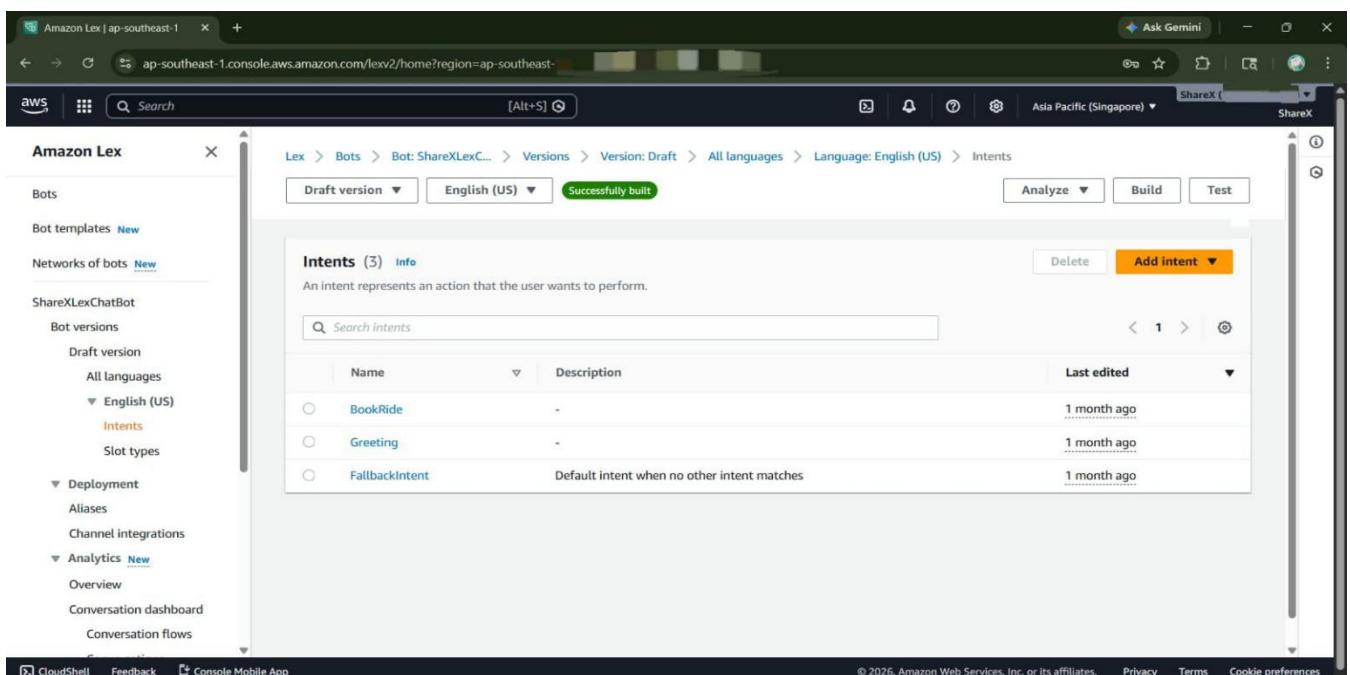


Figure 12. Amazon Lex – ShareXLexChatBot Intents

Cost Management The entire ShareX project is deployed using AWS Free Tier services, ensuring minimal operational cost. As per the latest billing data:

- Month-to-date cost: \$0.01
- Forecasted cost for the current month: \$0.16
- Previous month's total cost: \$0.07

Budget alerts are configured and currently show OK status with no cost anomalies. This demonstrates the high cost-efficiency of the serverless architecture, making it highly suitable for academic and small-scale deployments.

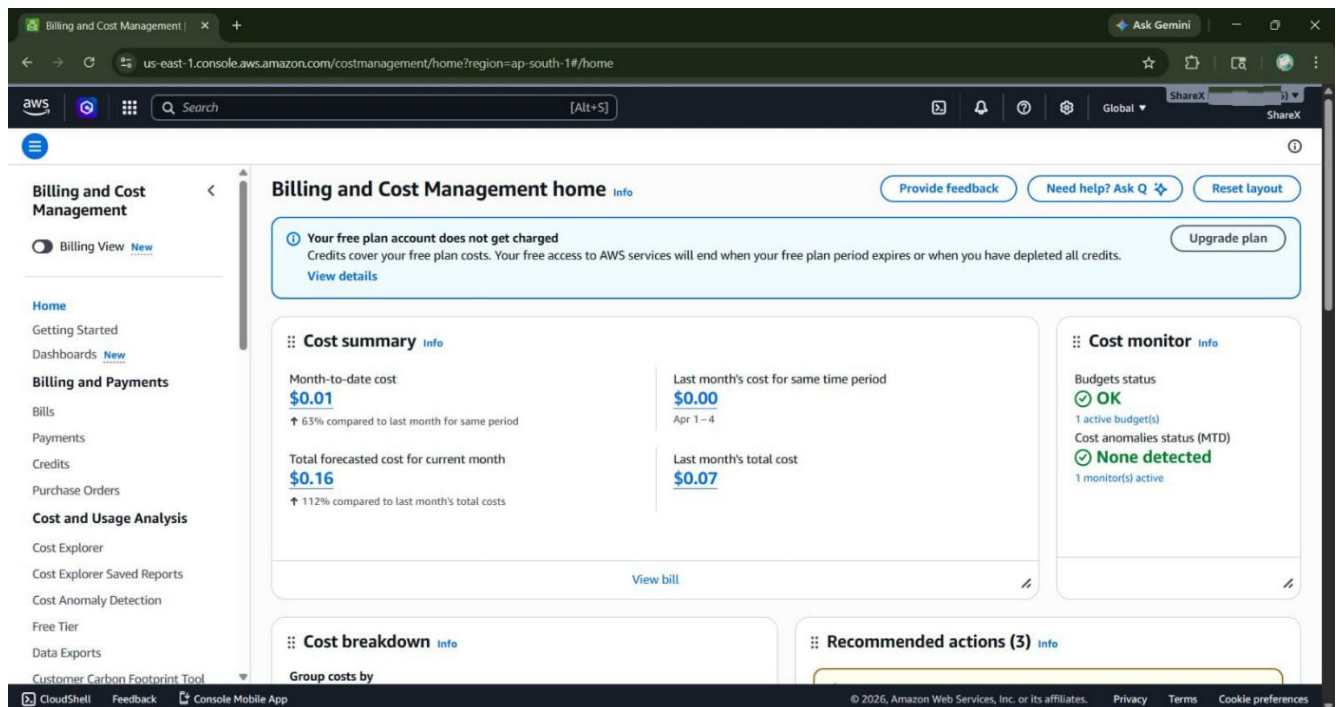


Figure 13. AWS Billing and Cost Management – Free Tier Usage

CHAPTER 5 - RESULTS AND DISCUSSION

5.1 Implementation Summary

The ShareX ride-sharing platform was successfully implemented using a fully serverless architecture on Amazon Web Services (AWS). The system effectively achieved the defined functional and non-functional requirements, delivering a modern, responsive, and secure ride-sharing experience.

The frontend was developed using HTML5, Tailwind CSS, and JavaScript and was successfully hosted on AWS Amplify with automated CI/CD integration from GitHub. The backend logic was implemented through AWS Lambda functions, while Amazon API Gateway managed all REST API communications. User authentication and authorization were securely handled by Amazon Cognito, and ride data was stored in Amazon DynamoDB. Additionally, Amazon Lex was integrated to provide a conversational chatbot interface for ride booking.

Key features such as user registration, login, ride request booking, ride history, and chatbot interaction are fully functional. The application demonstrated good performance with average API response times between 200–400 ms under normal load. The serverless design enabled automatic scaling and high availability without manual infrastructure management.

5.2 Challenges Faced and Resolutions

During the development and deployment of ShareX, several challenges were encountered and successfully resolved:

- **Cold Start Latency in AWS Lambda:** Initial delays occurred when Lambda functions were invoked after periods of inactivity. This was mitigated by optimizing code size, reducing unnecessary dependencies, and configuring appropriate memory allocation.
- **Cognito and API Gateway Authentication Issues:** Early integration between Amazon Cognito and API Gateway resulted in frequent authorization errors. This was resolved by correctly configuring the Cognito Authorizer in API Gateway and properly handling JWT token validation.
- **DynamoDB Query Optimization:** Inefficient data retrieval, especially for ride history, was initially an issue. This was addressed by designing proper partition keys and using Query operations instead of Scan operations.
- **Amazon Lex Chatbot Accuracy:** The chatbot initially struggled with ambiguous user inputs. This was improved by refining intents, adding more training phrases, and enhancing slot configurations.
- **CI/CD Pipeline Issues:** Deployment failures occurred due to incorrect build settings in AWS Amplify. These were fixed by updating the build specification and correctly managing environment variables in GitHub Secrets.

5.3 Performance and Cost Observations

Performance Analysis

The system exhibited satisfactory performance across all major components. Most backend operations handled by AWS Lambda completed within **200–400 ms**. DynamoDB queries showed excellent response times, often under **100 ms** for optimized queries. Amazon Cognito authentication typically completed within 1–2 seconds, while the Amazon Lex chatbot responded in approximately **2–4 seconds** depending on input complexity.

The serverless architecture proved effective in handling concurrent requests through automatic scaling of Lambda and API Gateway, maintaining consistent performance.

Cost Analysis

One of the major achievements of the project is its high cost efficiency. The entire application was developed and deployed within the **AWS Free Tier** limits:

- Month-to-date cost remained extremely low (approximately **\$0.01**).
- The serverless model (Lambda + API Gateway + DynamoDB on-demand) ensured near-zero cost during idle periods.
- AWS Amplify provided free hosting and CI/CD capabilities.
- No additional charges were incurred for compute, storage, or authentication services under normal usage.

The pay-as-you-go nature of AWS serverless services proved highly beneficial for this academic project, eliminating the need for provisioning and maintaining dedicated servers.

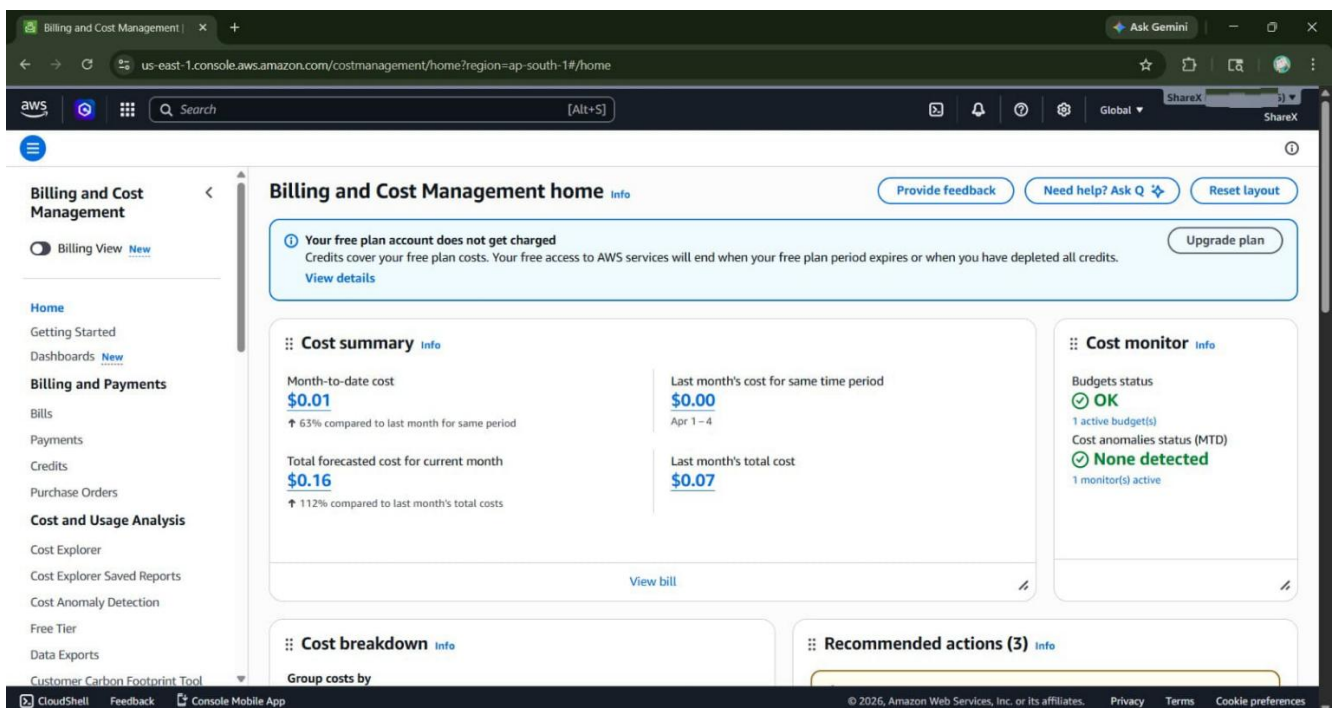


Figure 7. Daily Cost Graph

5.1 KEY LEARNINGS AND TEAM CONTRIBUTIONS

The project was executed using a collaborative approach. Tasks were divided among team members covering frontend development, backend logic, database design, authentication, chatbot integration, and testing. A GitHub branching strategy was followed, where each member worked on separate feature branches and used pull requests for code review before merging into the main branch. This practice ensured better code quality and minimized conflicts.

Regular team meetings, integration testing, and code reviews played a vital role in successful module integration.

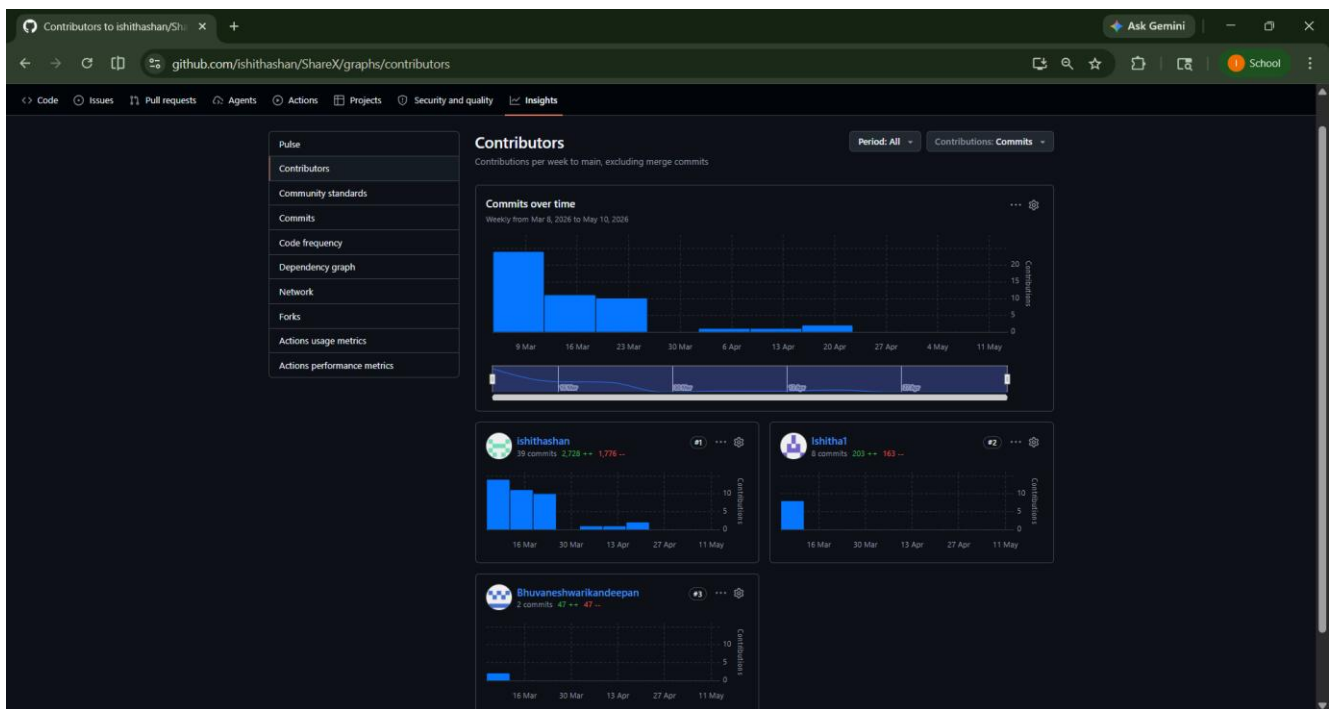


Figure 8. Github Contributions

CHAPTER 6 - CONCLUSION AND FUTURE WORKS

6.1 CONCLUSION

The ShareX ride-sharing platform was successfully developed as a secure, scalable, and user-friendly web application using modern AWS serverless technologies. The project effectively addressed key challenges faced by commuters such as unreliable ride availability, poor user experience, and lack of transparency in traditional transportation systems.

By leveraging services like AWS Amplify, AWS Lambda, Amazon API Gateway, Amazon Cognito, Amazon DynamoDB, and Amazon Lex, the system delivers a complete ride-booking experience with secure authentication, real-time interaction, and an intuitive interface. The adoption of a serverless architecture not only reduced infrastructure management overhead but also ensured automatic scaling and high cost efficiency.

The project demonstrated a strong implementation of cloud-native development practices, including CI/CD pipelines, secure API development, and conversational AI integration. From a learning perspective, it provided invaluable practical exposure to modern cloud computing, DevSecOps principles, and full-stack application development.

Overall, ShareX stands as a successful demonstration of how serverless technologies can be used to build efficient, secure, and accessible digital solutions for real-world transportation problems.

6.2 FUTURE WORKS

The current ShareX platform provides a strong and scalable foundation for further enhancements. While the existing system successfully delivers core ride-booking functionality, several advanced features can be implemented to make the platform more intelligent, efficient, and user-friendly.

Short-to-Medium Term Enhancements

- **Smart Ride Recommendation System:** An AI-powered recommendation engine can be developed to suggest optimal rides based on user preferences, past travel history, and frequently visited locations.
- **Real-Time Ride Tracking:** Integration of GPS-based real-time tracking will allow users to monitor driver location, estimated arrival time, and live trip progress, significantly improving transparency and safety.
- **Ride Pooling / Shared Rides:** The system can be enhanced to support ride-sharing among multiple passengers traveling in similar routes, thereby reducing costs and optimizing vehicle utilization.

- **Advanced Payment Integration:** Integration with digital wallets, UPI, and automated fare splitting features can be added for seamless transactions.

Long-Term Vision

- **Multi-City and Multi-Region Expansion:** The architecture can be scaled to support operations across multiple cities and regions by improving data management and geolocation capabilities.
- **Advanced AI and Machine Learning:** Integration of more sophisticated AI services for demand forecasting, dynamic pricing, anomaly detection, and predictive analytics.
- **Mobile Application Development:** A dedicated Android and iOS mobile application will be developed to provide better accessibility, push notifications, and an enhanced user experience compared to the web platform.
- **Driver Analytics Dashboard:** A separate dashboard for drivers can be introduced to track earnings, performance, and route efficiency.

These future enhancements will transform ShareX from a functional ride-booking platform into a comprehensive, intelligent, and widely accessible mobility solution.

APPENDICES

Appendix A: Project Screenshots

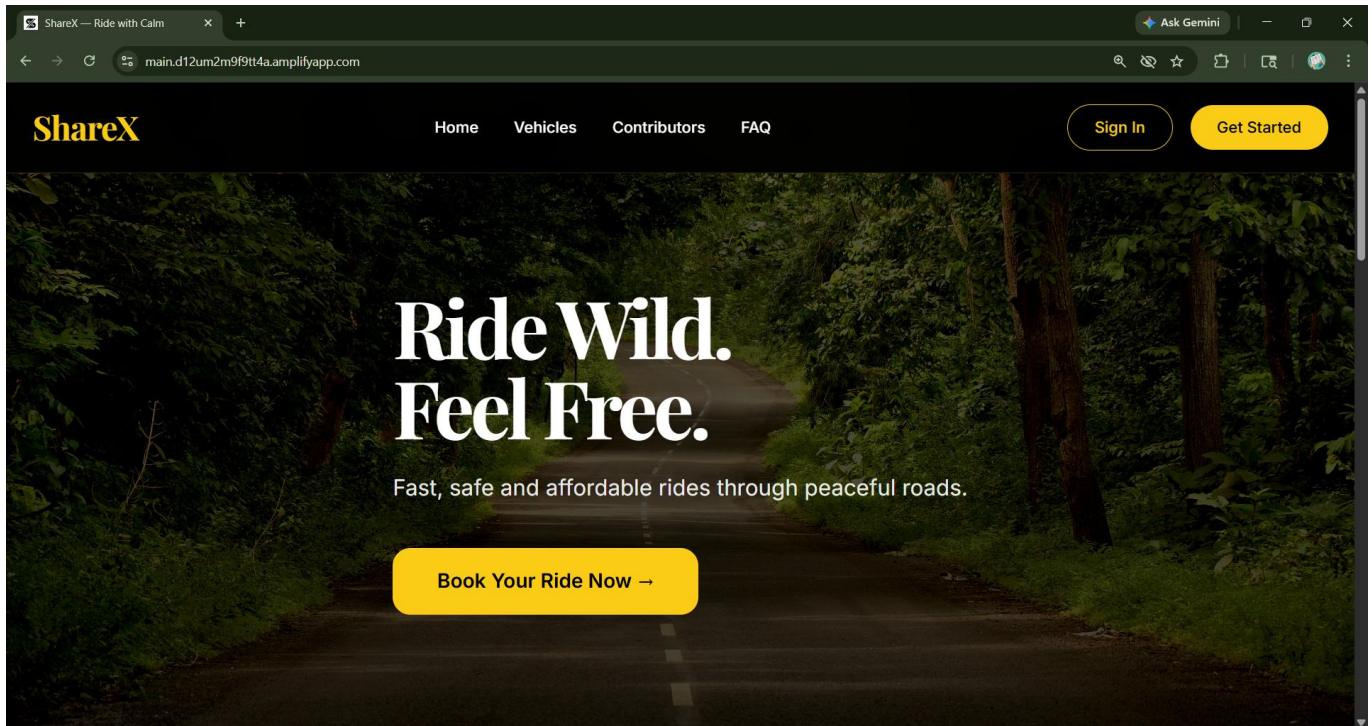


Figure 1: ShareX Homepage / Hero Section

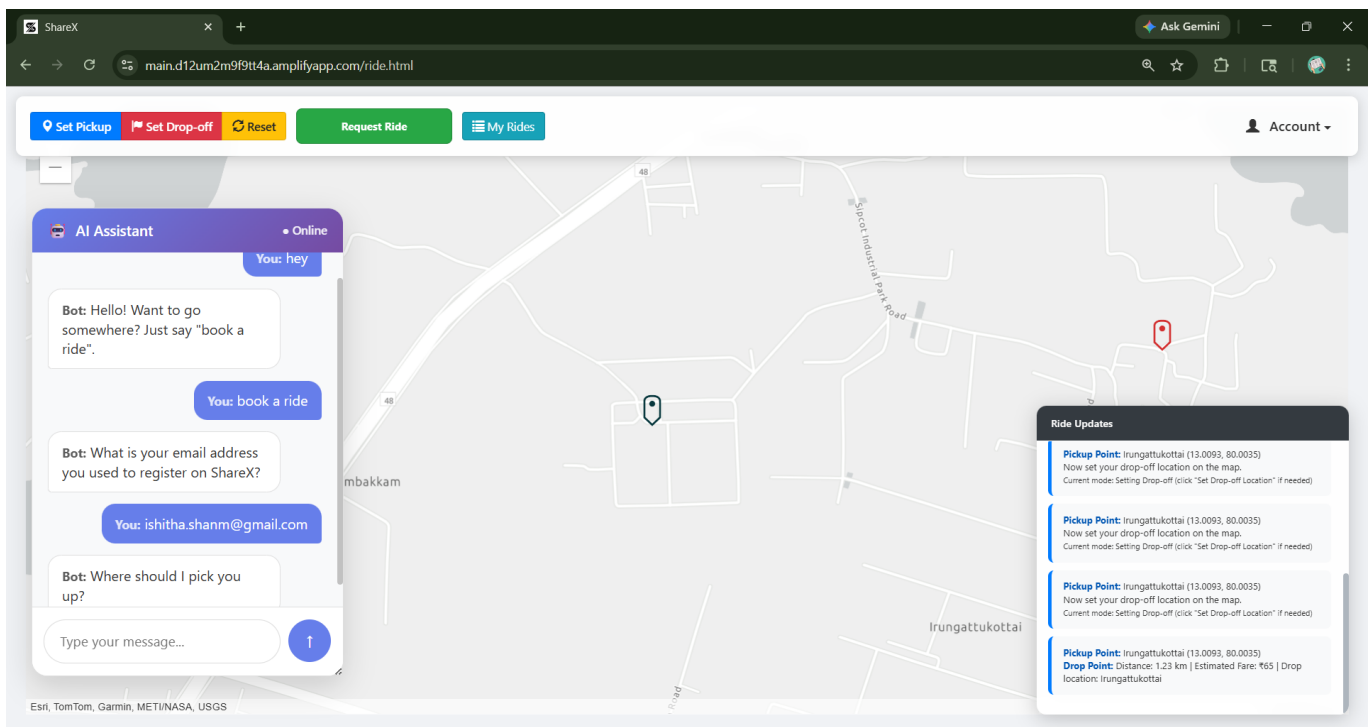


Figure 2: Ride Booking Interface + Amazon Lex Chatbot Interface

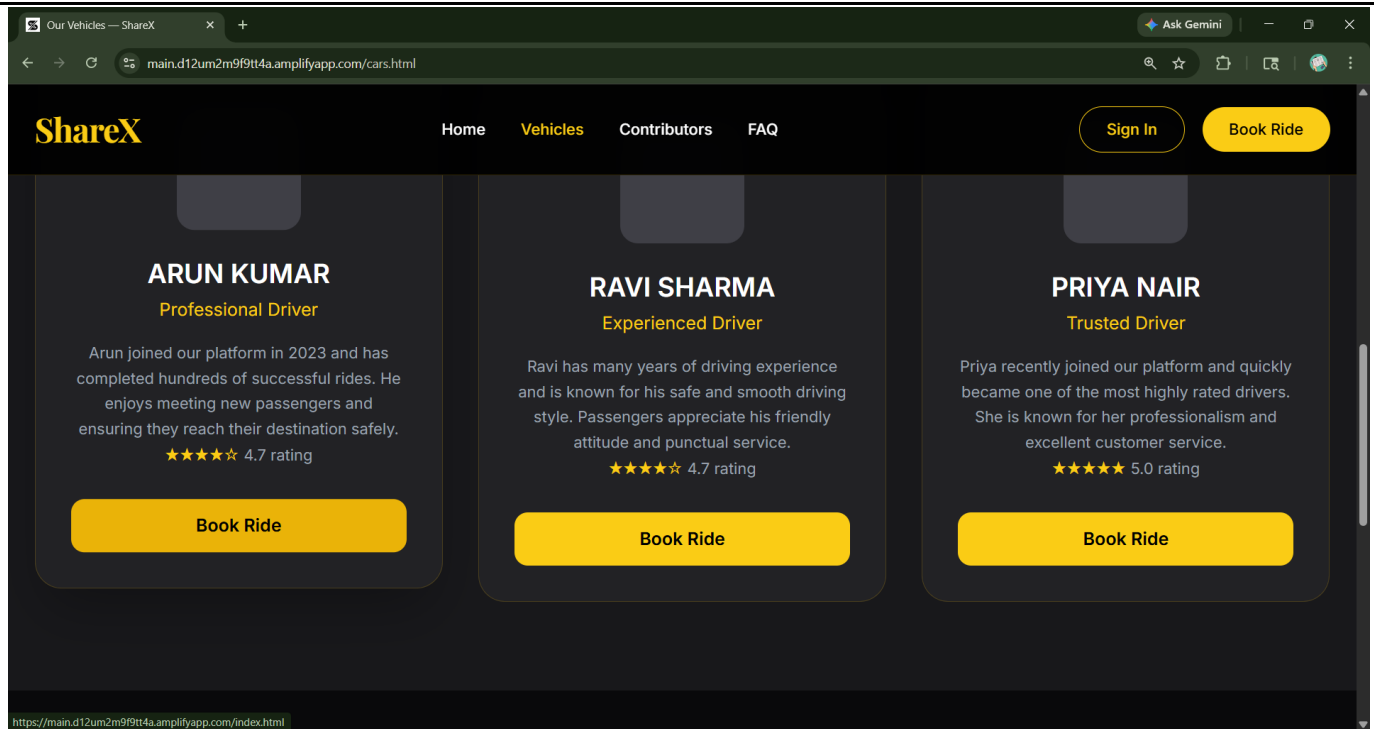


Figure 3: Available Cars / Vehicles Page

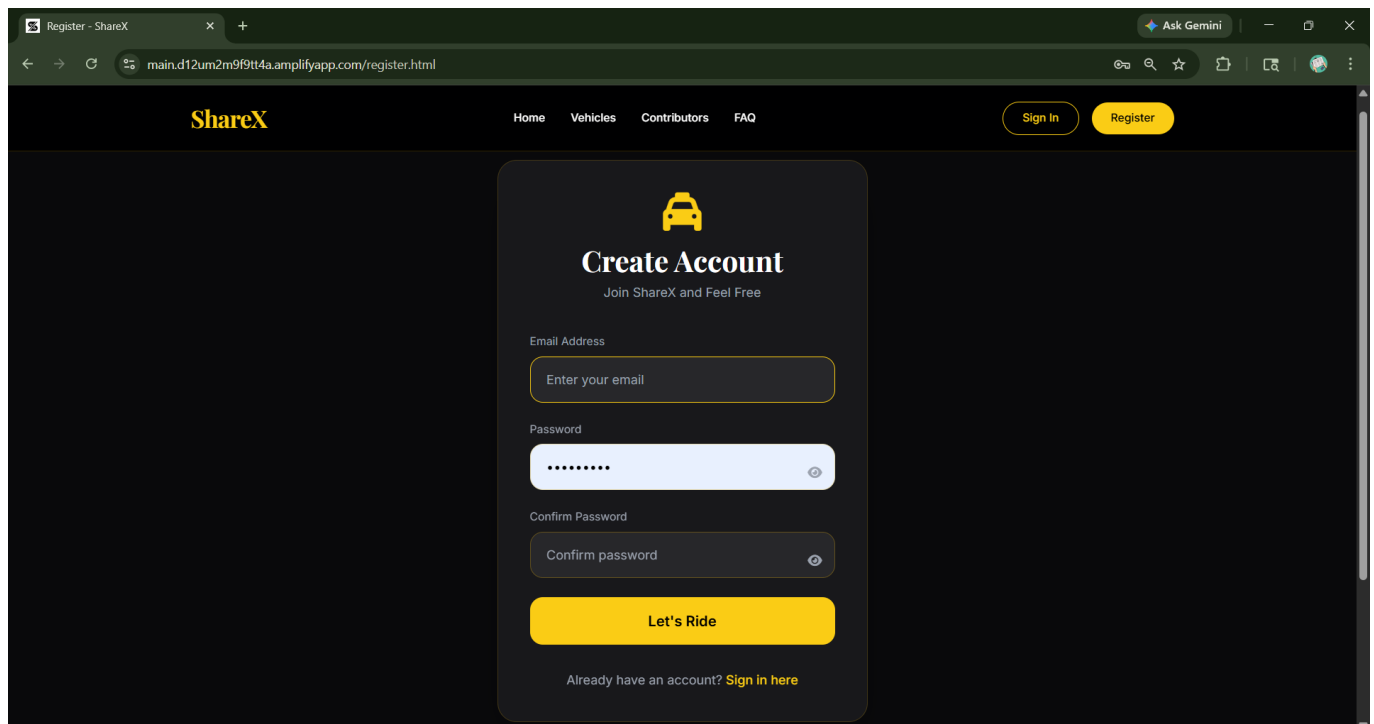


Figure 4: User Registration and Login Pages (Cognito)

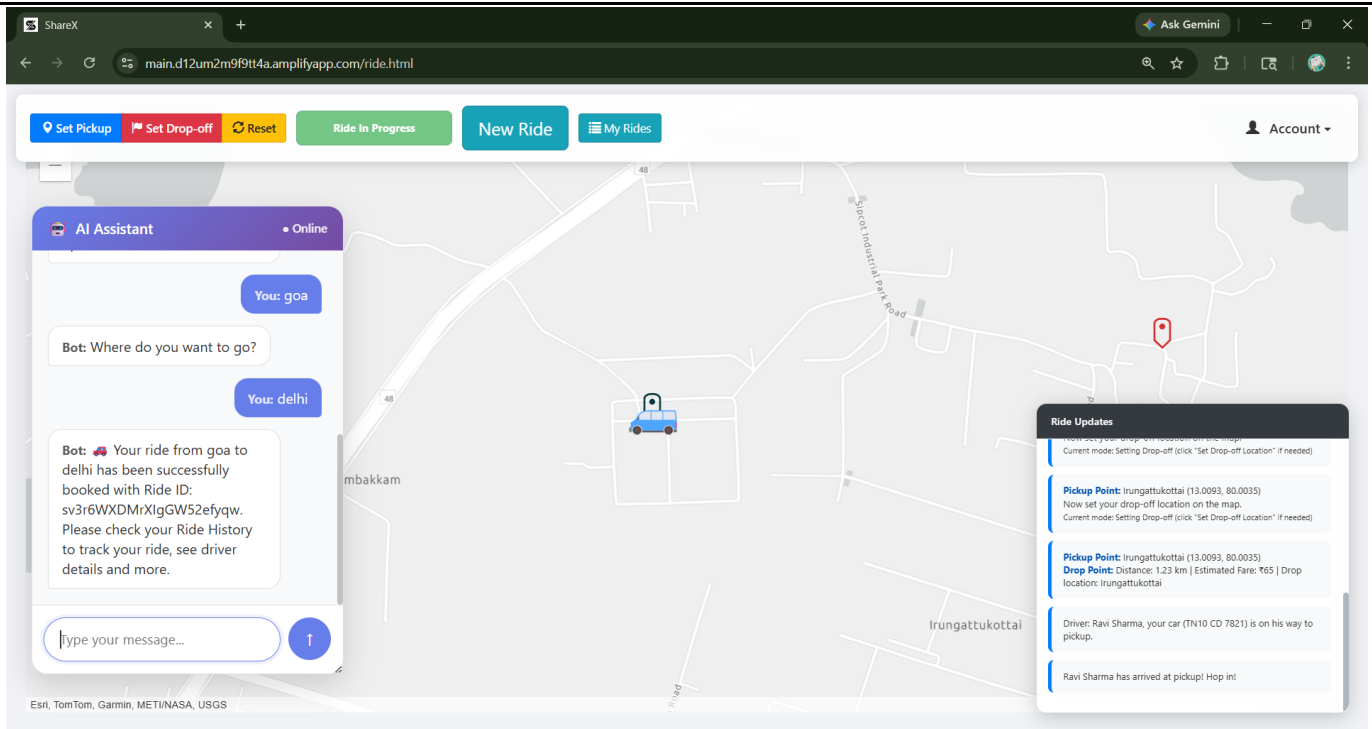


Figure 5: Successful Ride Booking Confirmation

Appendix B: AWS Console Screenshots

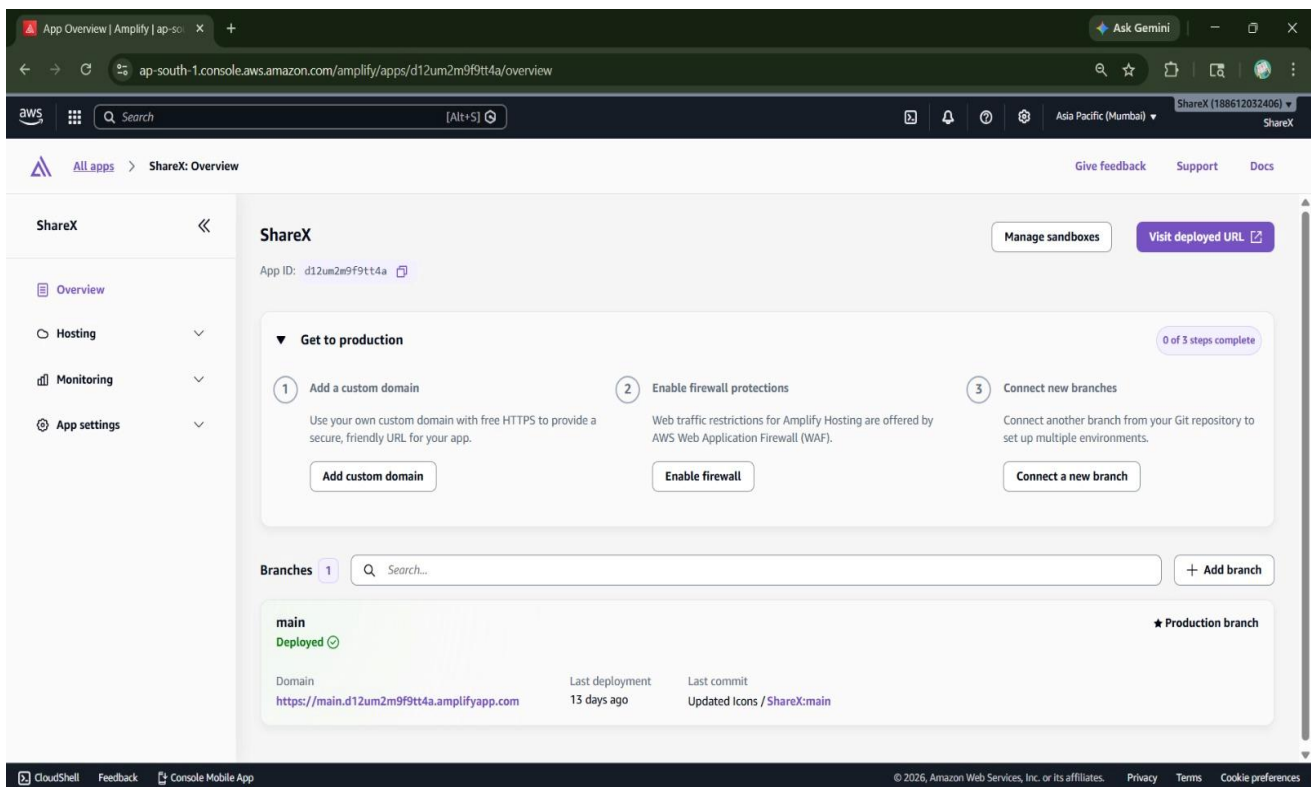


Figure 8: AWS Amplify Hosting Dashboard

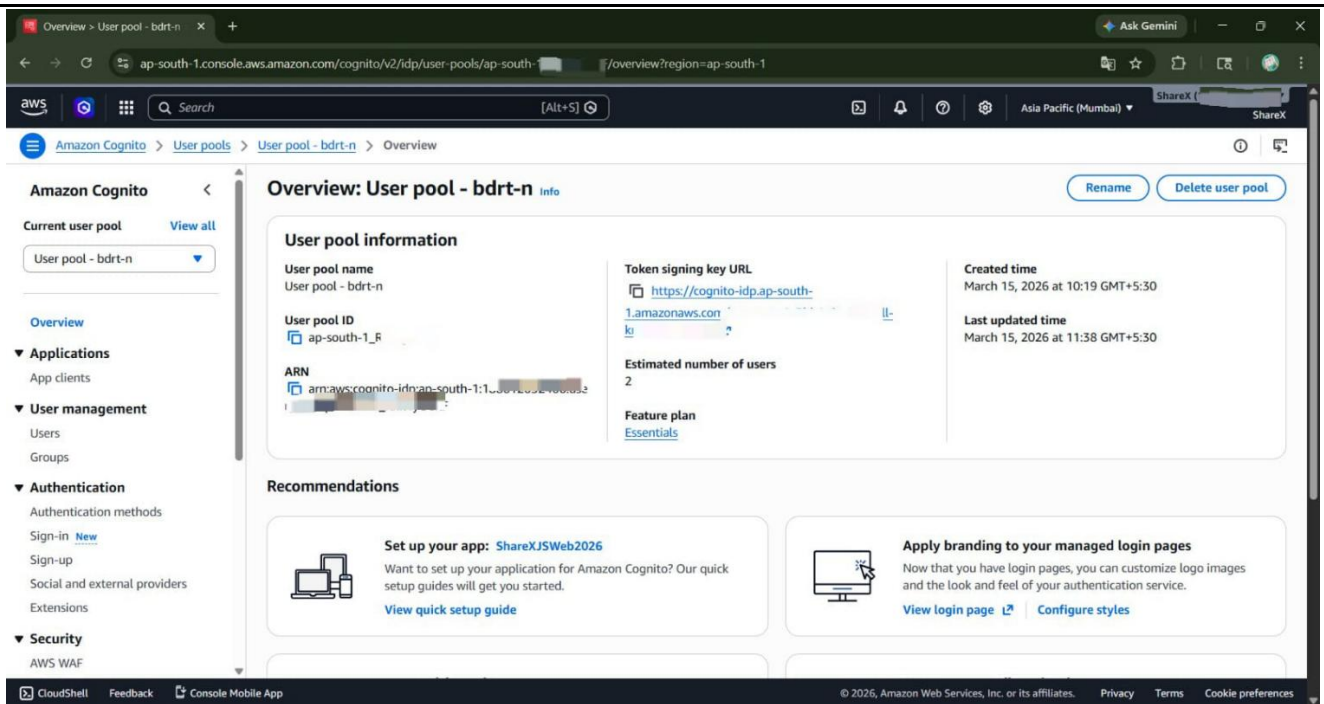


Figure 9: Amazon Cognito User Pool Configuration

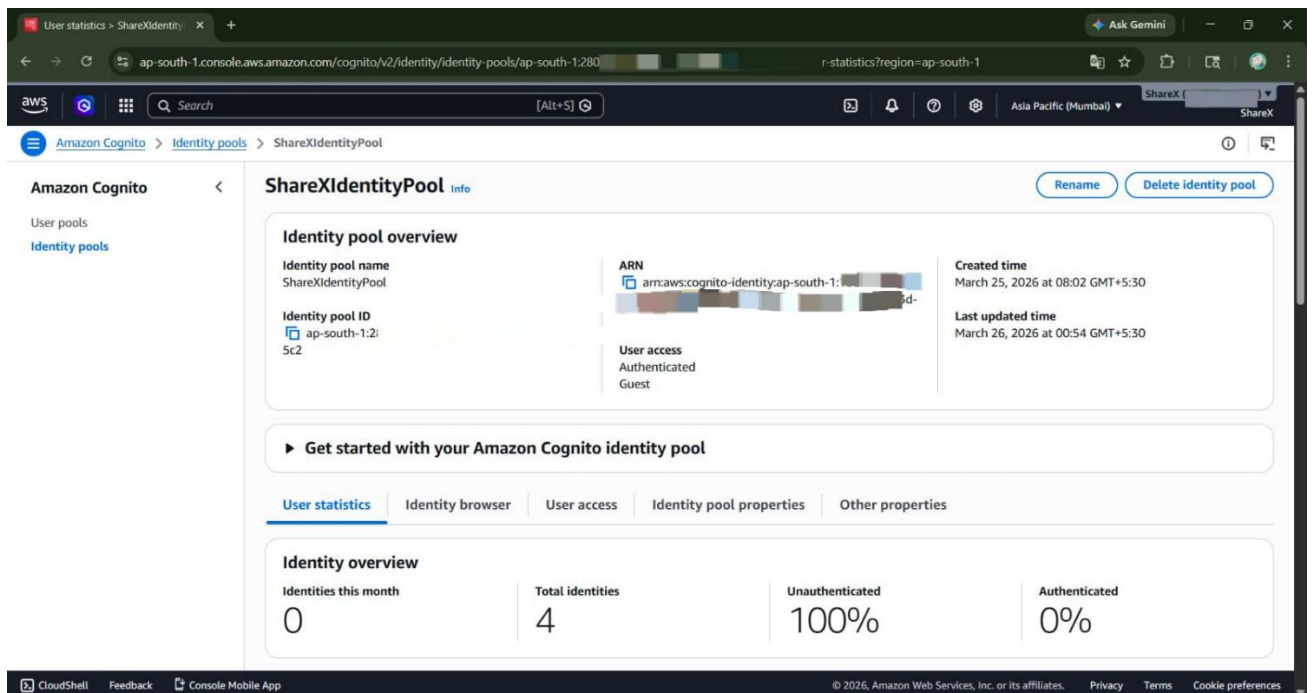


Figure 10: Amazon Cognito Identity Pool

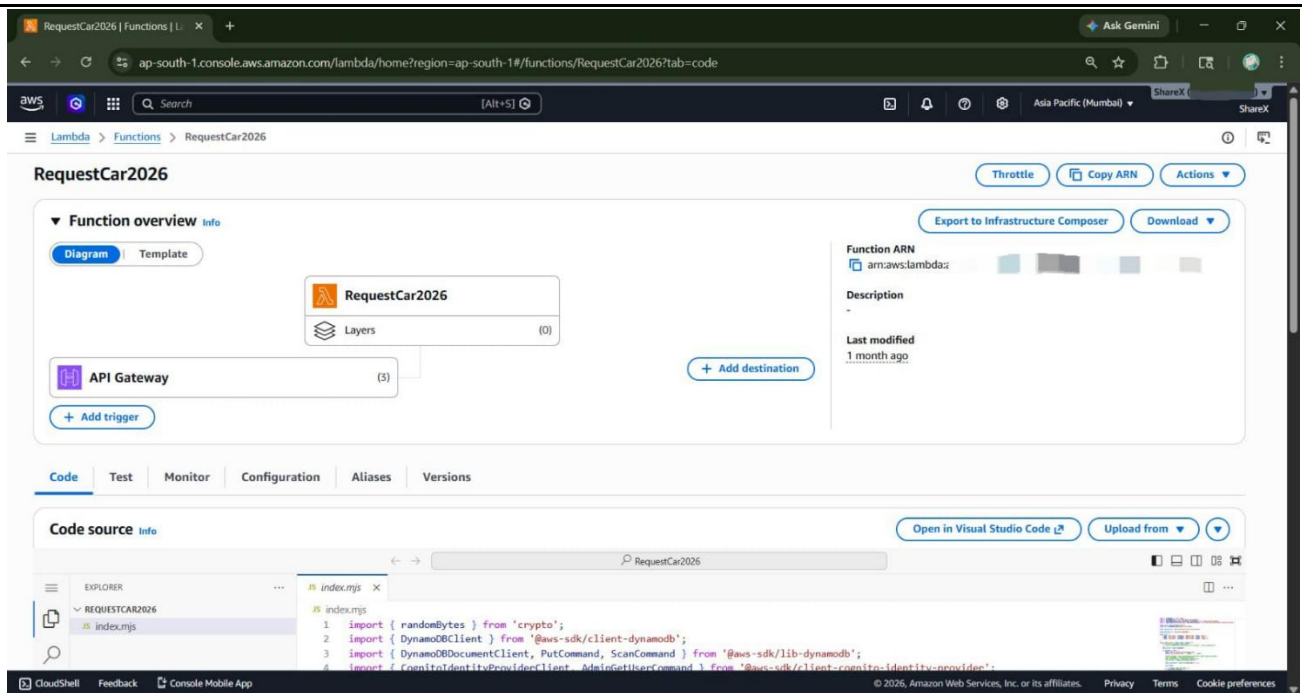


Figure 11: AWS Lambda Function (RequestCar2026)

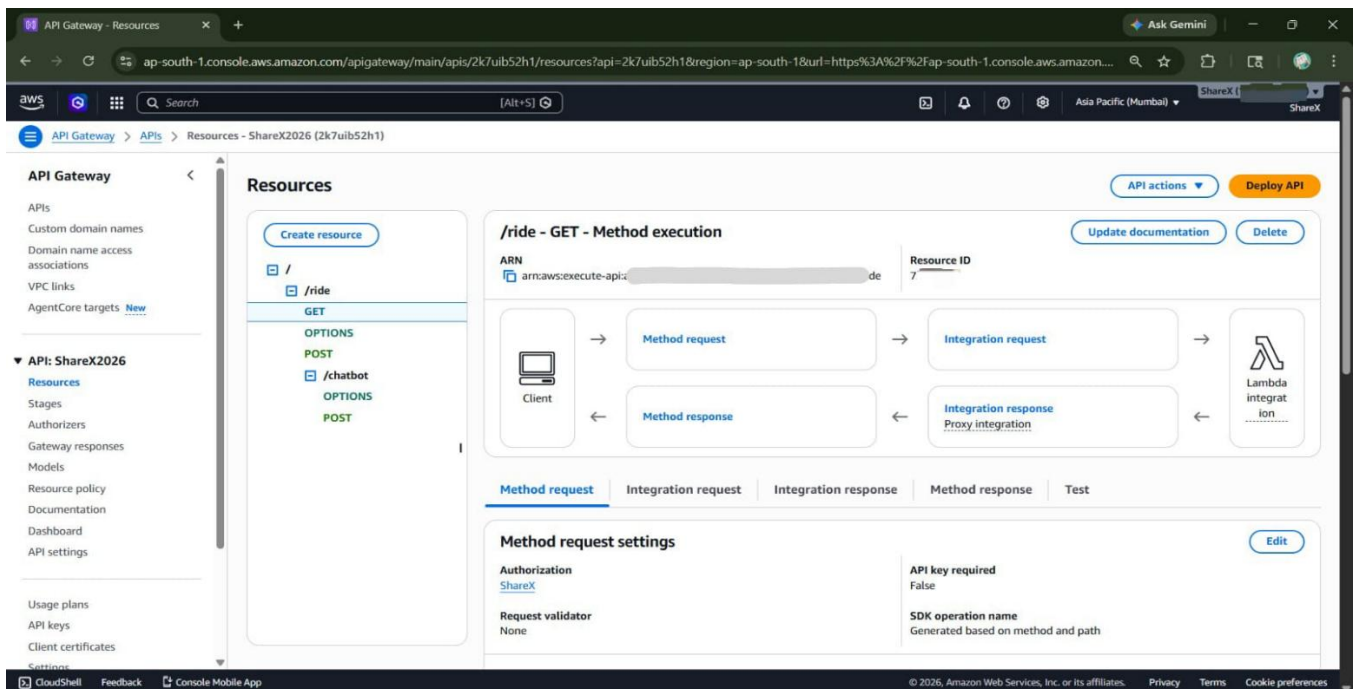


Figure 12: Amazon API Gateway (ShareX2026) Resources

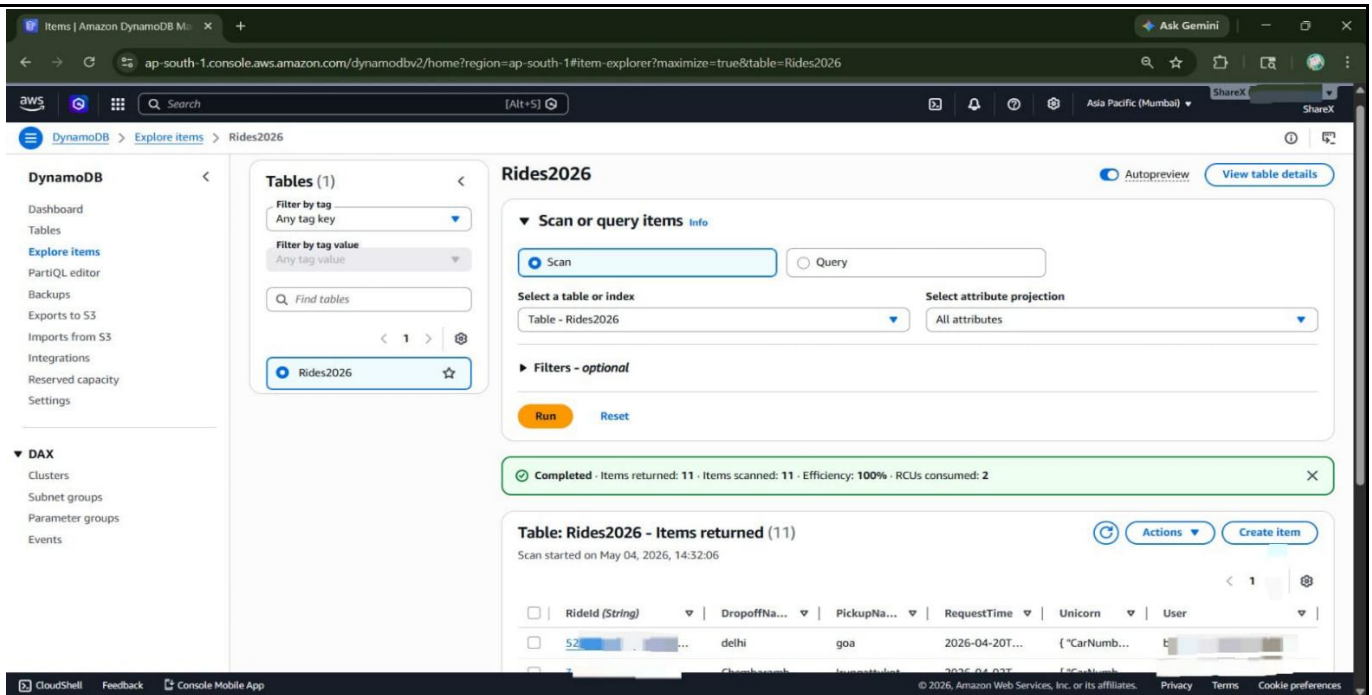


Figure 13: Amazon DynamoDB Rides2026 Table

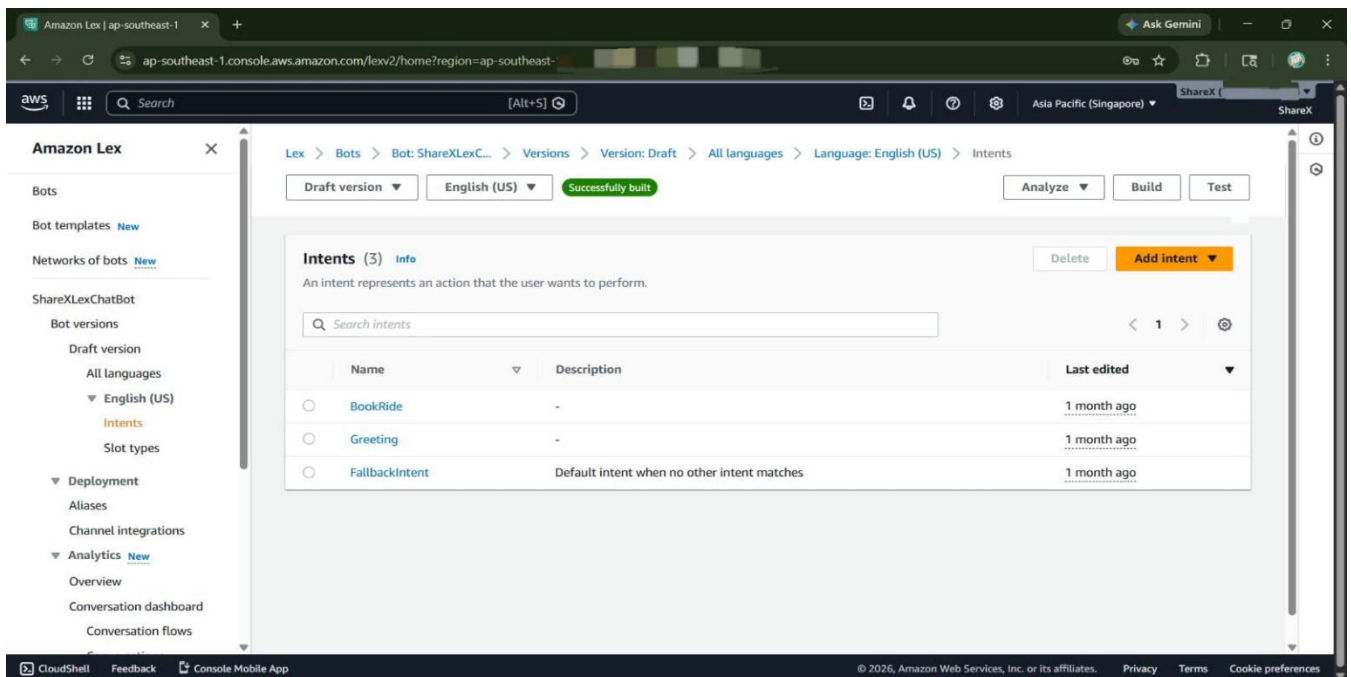


Figure 14: AWS IAM Roles Created for the Project

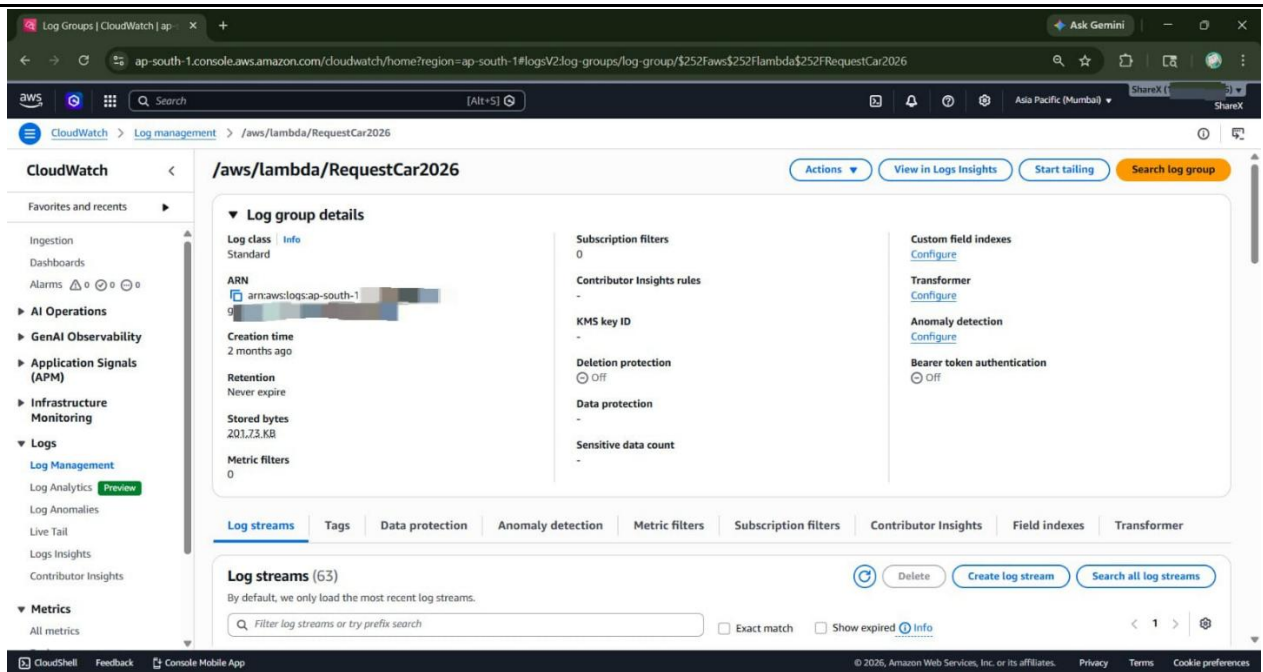


Figure 15: Amazon CloudWatch Logs

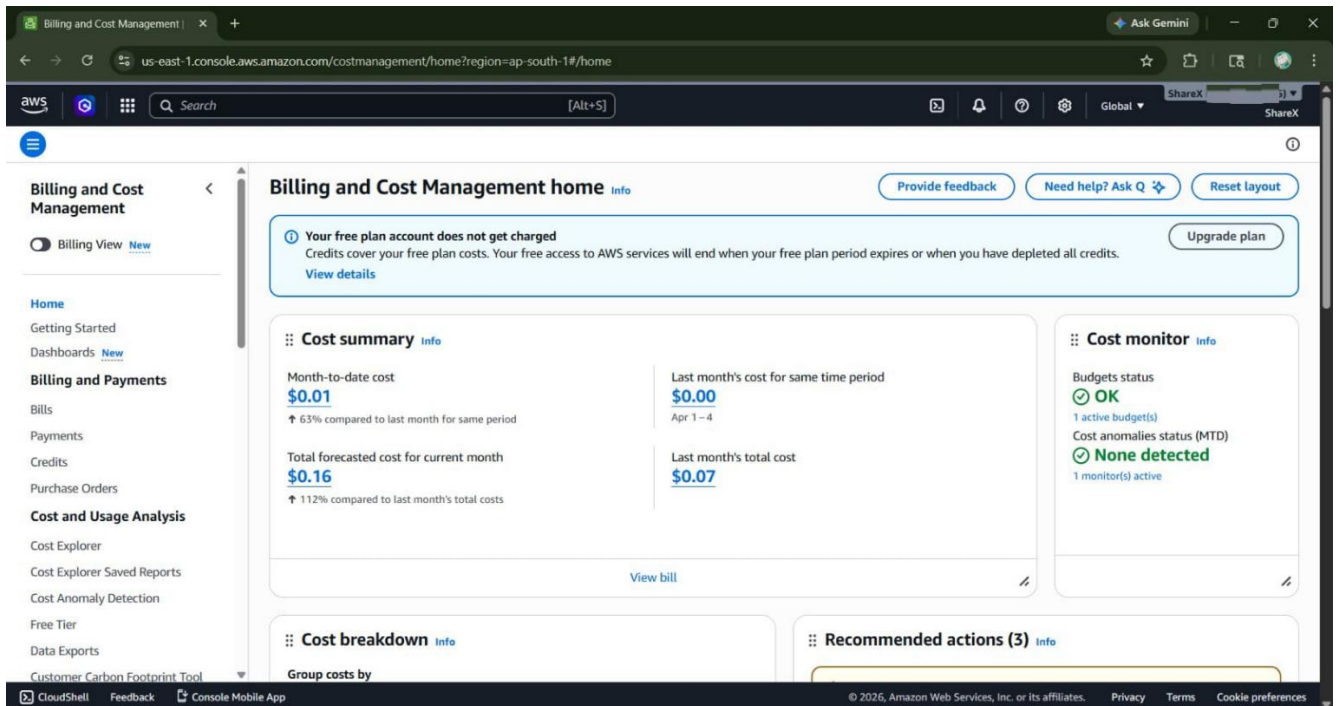


Figure 16: AWS Billing & Cost Management (Free Tier Usage)

Appendix C: Code Samples

```
import { randomBytes } from 'crypto';
import { DynamoDBClient } from '@aws-sdk/client-dynamodb';
import { DynamoDBDocumentClient, PutCommand, ScanCommand } from '@aws-sdk/lib-dynamodb';
import { CognitoIdentityProviderClient, AdminGetUserCommand } from '@aws-sdk/client-cognito-identity-provider';

const client = new DynamoDBClient({});
const ddb = DynamoDBDocumentClient.from(client);

const cognitoClient = new CognitoIdentityProviderClient({});

const USER_POOL_ID = "ap-south-1_Rhk4yOeKF";

const fleet = [
  { Name: 'Arun Kumar', CarNumber: 'TN09 AB 2345', Gender: 'Male' },
  { Name: 'Ravi Sharma', CarNumber: 'TN10 CD 7821', Gender: 'Male' },
  { Name: 'Priya Nair', CarNumber: 'TN07 EF 5564', Gender: 'Female' },
];

export const handler = async (event, context) => {

  const method = event.httpMethod;

  if (method === 'POST') {
    const rideId = toUrlString(randomBytes(16));
    console.log('Received event (', rideId, '): ', event);

    const requestBody = JSON.parse(event.body || '{}');

    let userSub;

    // === CHATBOT COMES WITH EMAIL ===
    if (requestBody.UserEmail) {
      try {
        const params = {
          UserPoolId: USER_POOL_ID,
          Username: requestBody.UserEmail.trim()
        };
        const command = new AdminGetUserCommand(params);
        const cognitoUser = await cognitoClient.send(command);

        const subAttr = cognitoUser.UserAttributes.find(attr => attr.Name === 'sub');
        userSub = subAttr ? subAttr.Value : null;

        console.log('Chatbot booking - Email:', requestBody.UserEmail, '- Sub:', userSub);
      } catch (cognitoErr) {
        console.error('Cognito AdminGetUser error:', cognitoErr);
        return errorResponse('Invalid email or user not found', context.awsRequestId);
      }
    }
    // === NORMAL WEBSITE BOOKING ===
    else if (event.requestContext?.authorizer?.claims) {
      userSub = event.requestContext.authorizer.claims['cognito:username']
        || event.requestContext.authorizer.claims['sub'];
      console.log('Website booking - UserSub from Cognito:', userSub);
    }
    else {
      return errorResponse('User identification required', context.awsRequestId);
    }

    if (!userSub) {
      return errorResponse('Could not identify user', context.awsRequestId);
    }

    const pickupName = requestBody.PickupName || "Unknown";
    const dropoffName = requestBody.DropoffName || "Unknown";
    const unicorn = findUnicorn(requestBody.PickupLocation || {});

    try {
      /*const pickupName = requestBody.PickupName || "Unknown";
      const dropoffName = requestBody.DropoffName || "Unknown";*/

      await recordRide(rideId, userSub, unicorn, pickupName, dropoffName);
      //await recordRide(rideId, username, unicorn);
      return {
        statusCode: 201,
        body: JSON.stringify({
          RideId: rideId,
          Unicorn: unicorn,
          PickupName: pickupName,
          DropoffName: dropoffName,
          Eta: '30 seconds',
          Rider: userSub,
        }),
        headers: { 'Access-Control-Allow-Origin': '*' },
      };
    }
  }
};
```

```

    } catch (err) {
        console.error(err);
        return ErrorResponse(err.message, context.awsRequestId);
    }
}
<same>
}
// Just make sure recordRide uses userSub as the 'User' field
async function recordRide(rideId, userSub, unicorn, pickupName, dropoffName) {
    const params = {
        TableName: 'Rides2026',
        Item: {
            RideId: rideId,
            User: userSub, // ← long UUID
            Unicorn: unicorn,
            RequestTime: new Date().toISOString(),
            PickupName: pickupName || "Unknown",
            DropoffName: dropoffName || "Unknown"
        },
    };
    await ddb.send(new PutCommand(params));
}
<same>

```

Figure 17: Sample AWS Lambda Function Code (RequestCar2026)

Appendix D: GitHub Repository Details

- Repository Link: <https://github.com/ishithashan/ShareX.git>